







































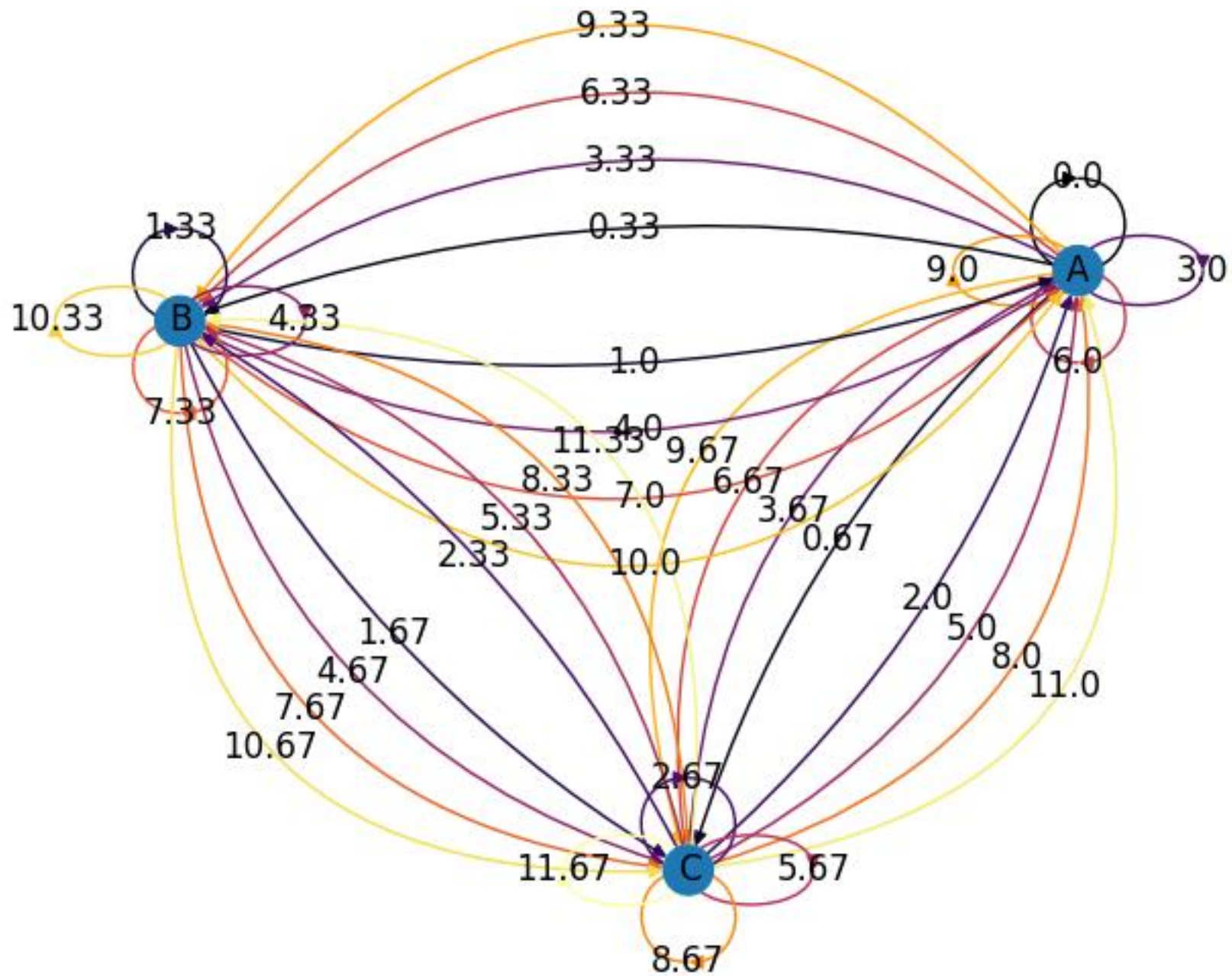


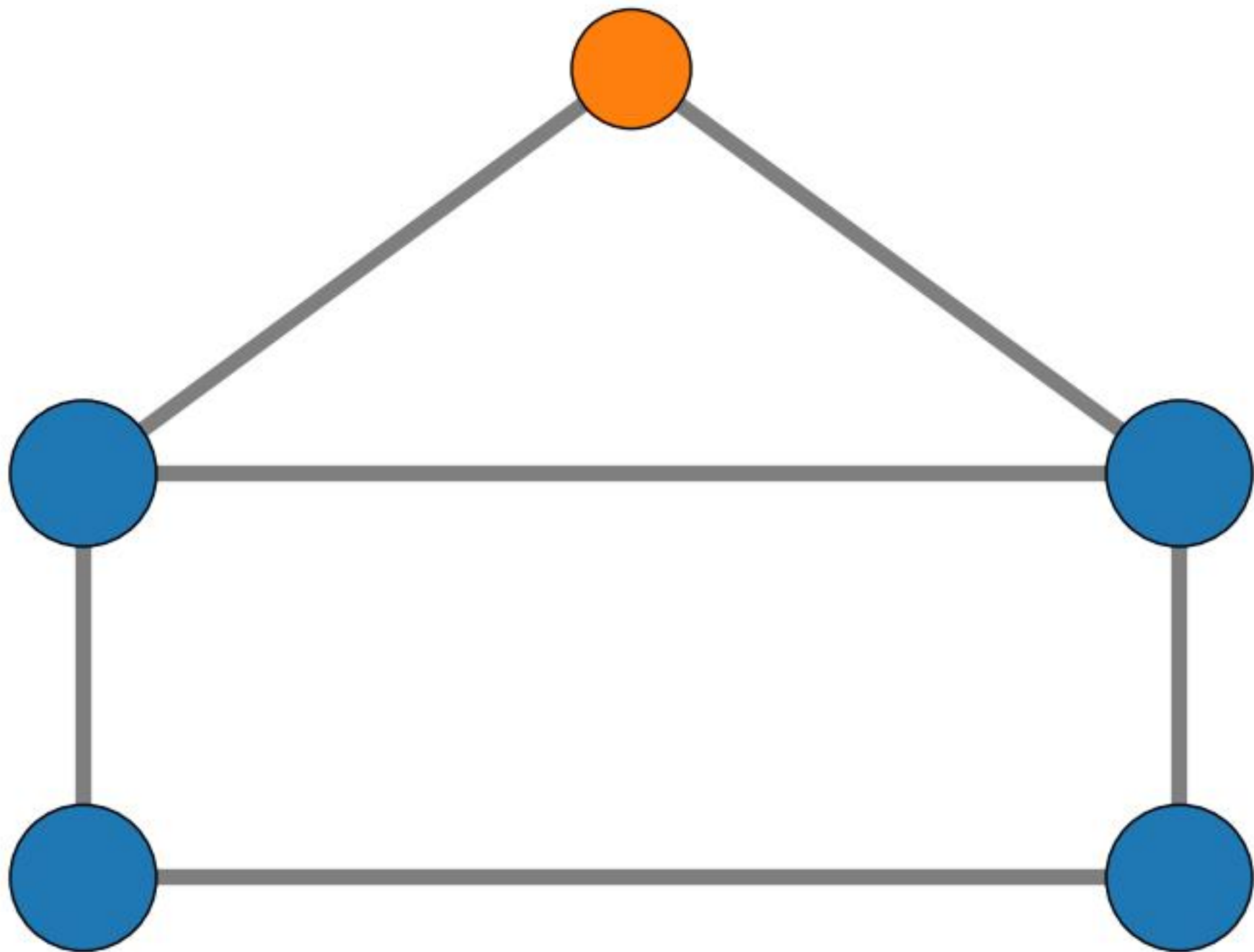


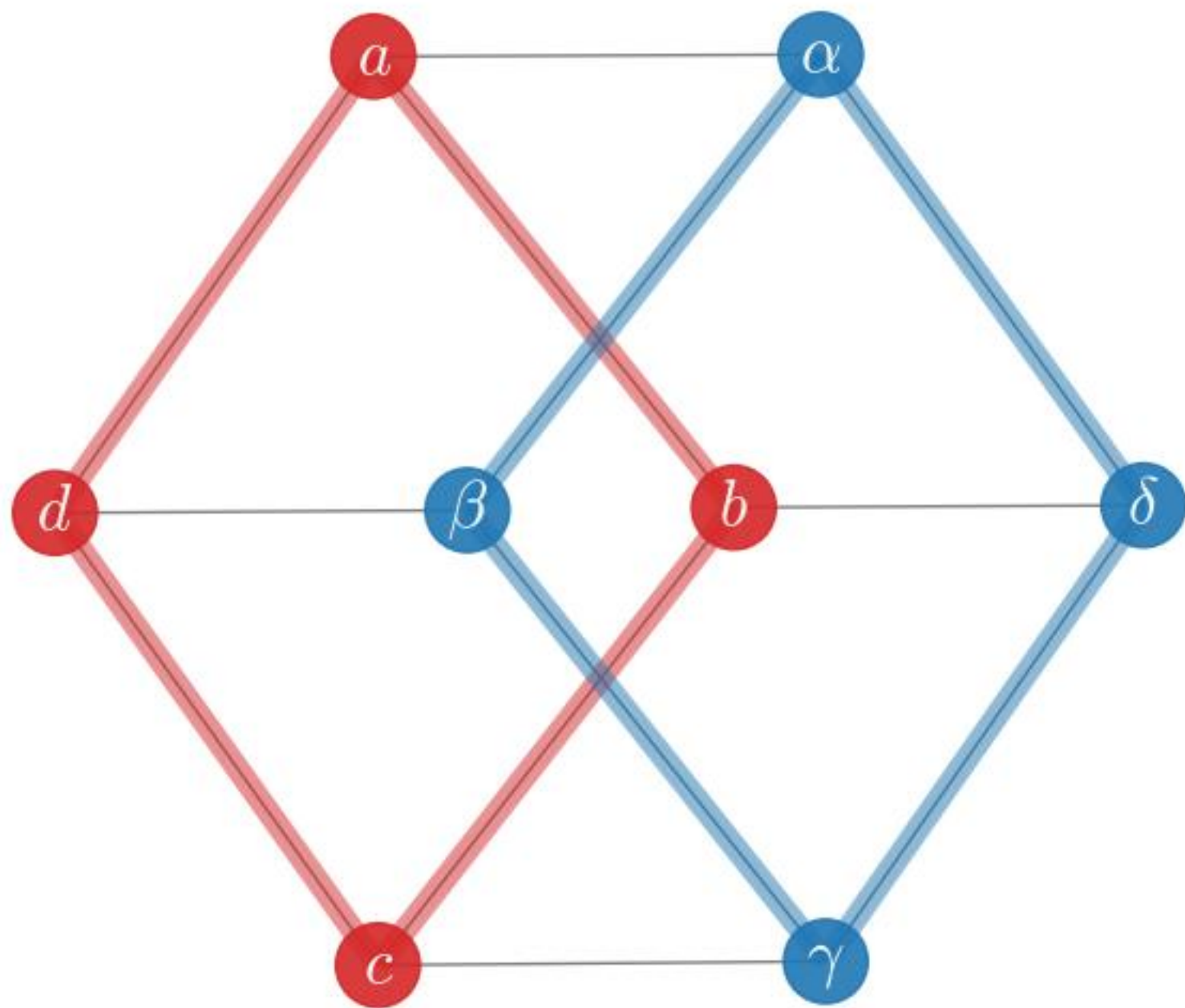


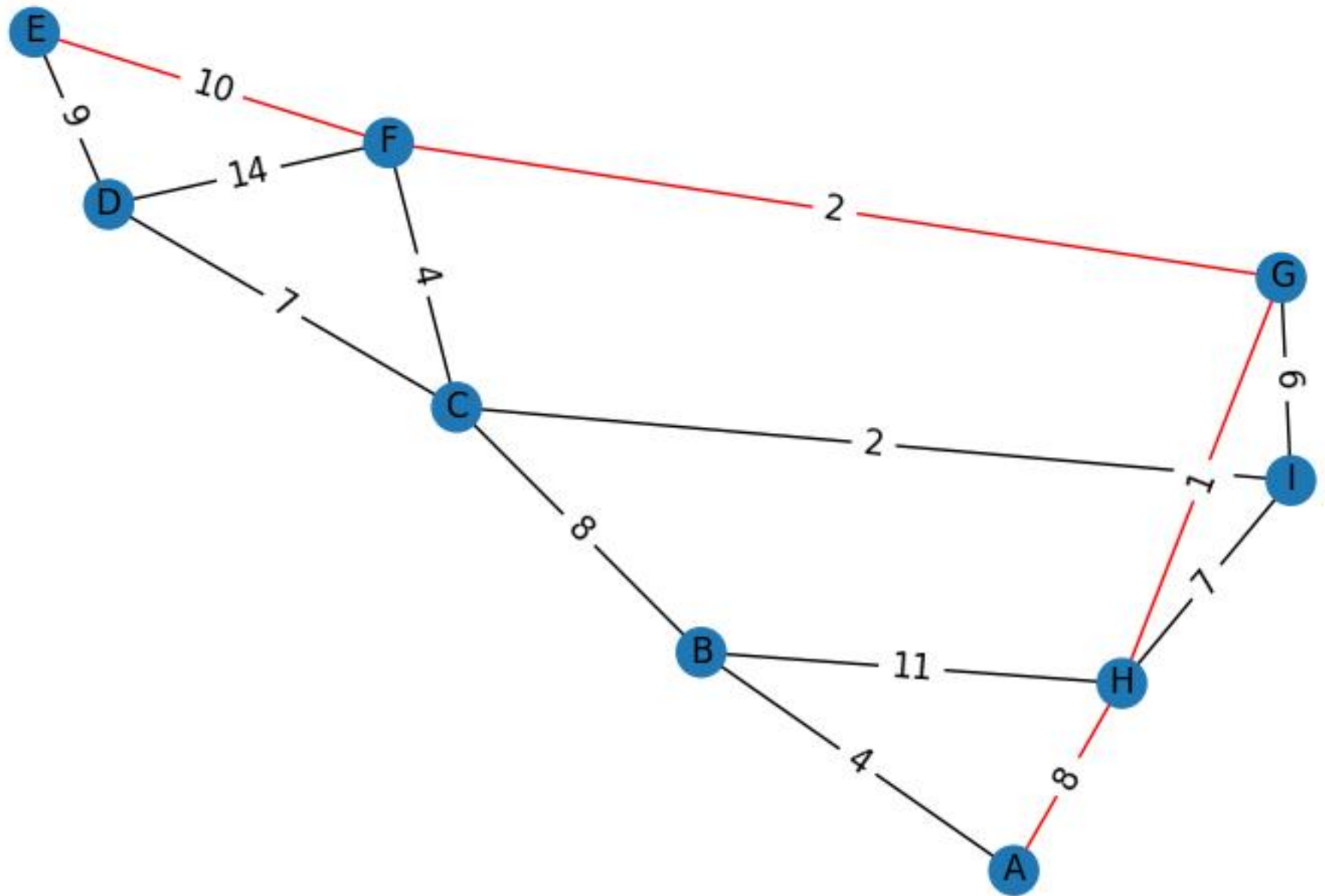


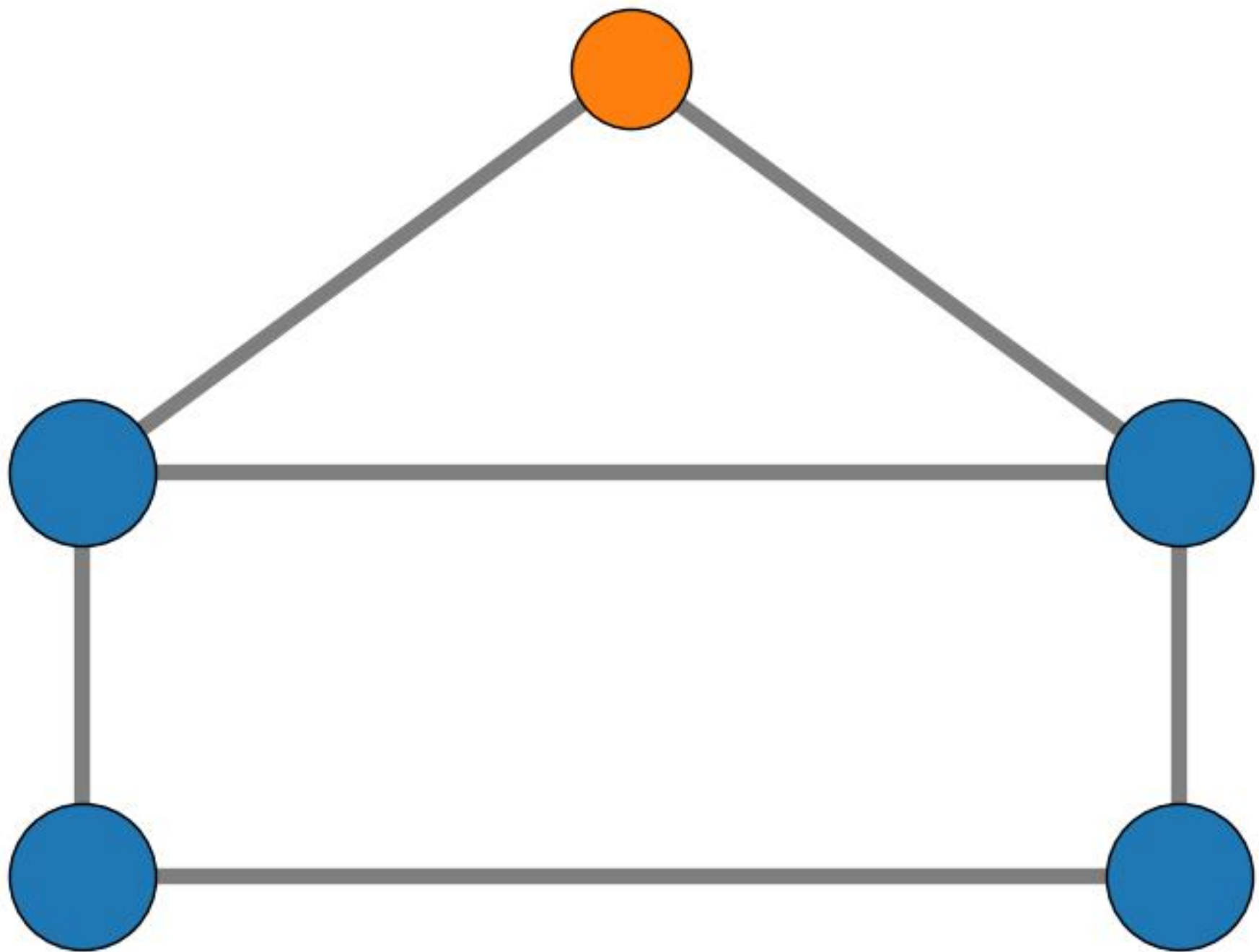
matplotlib







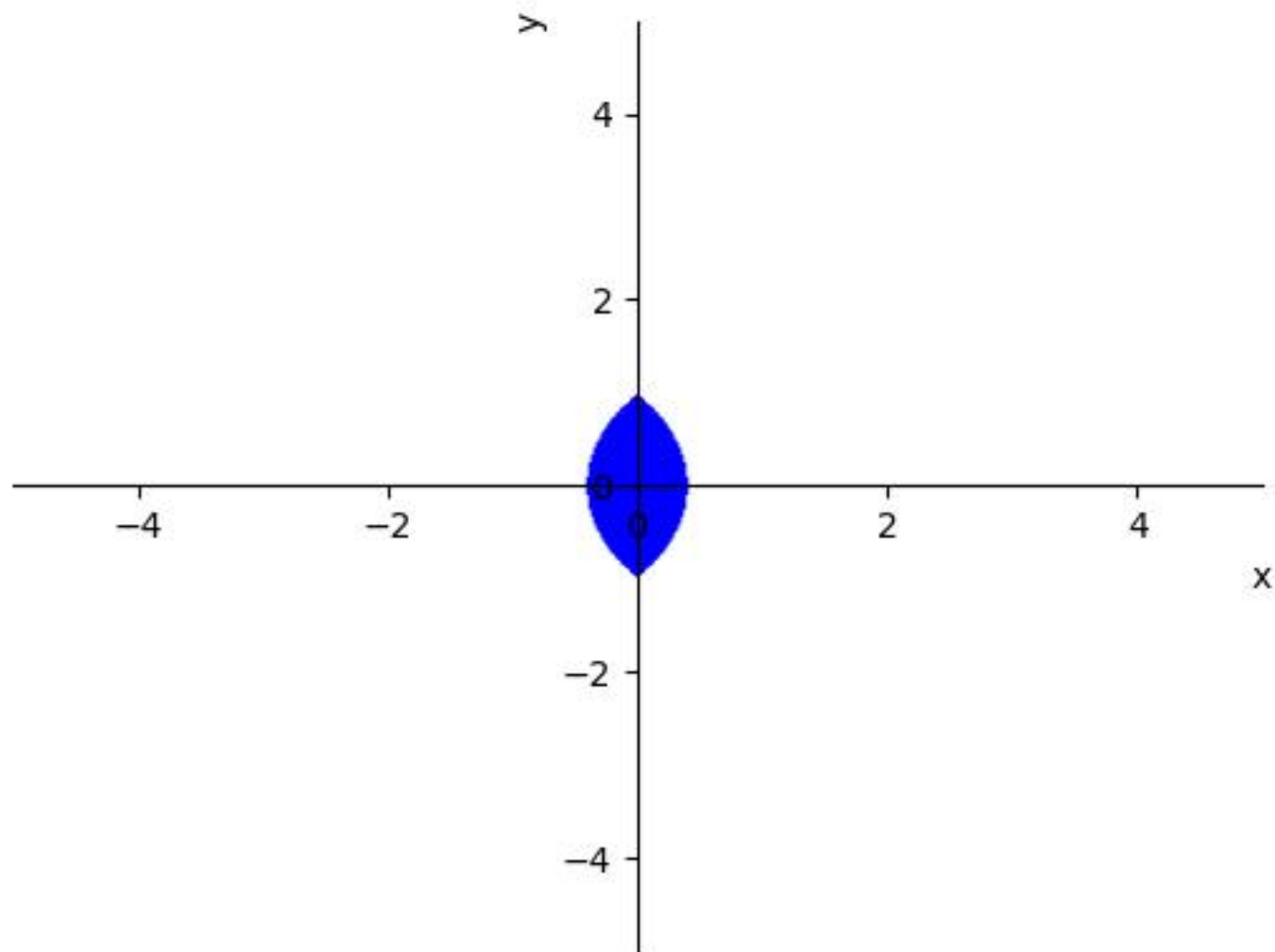


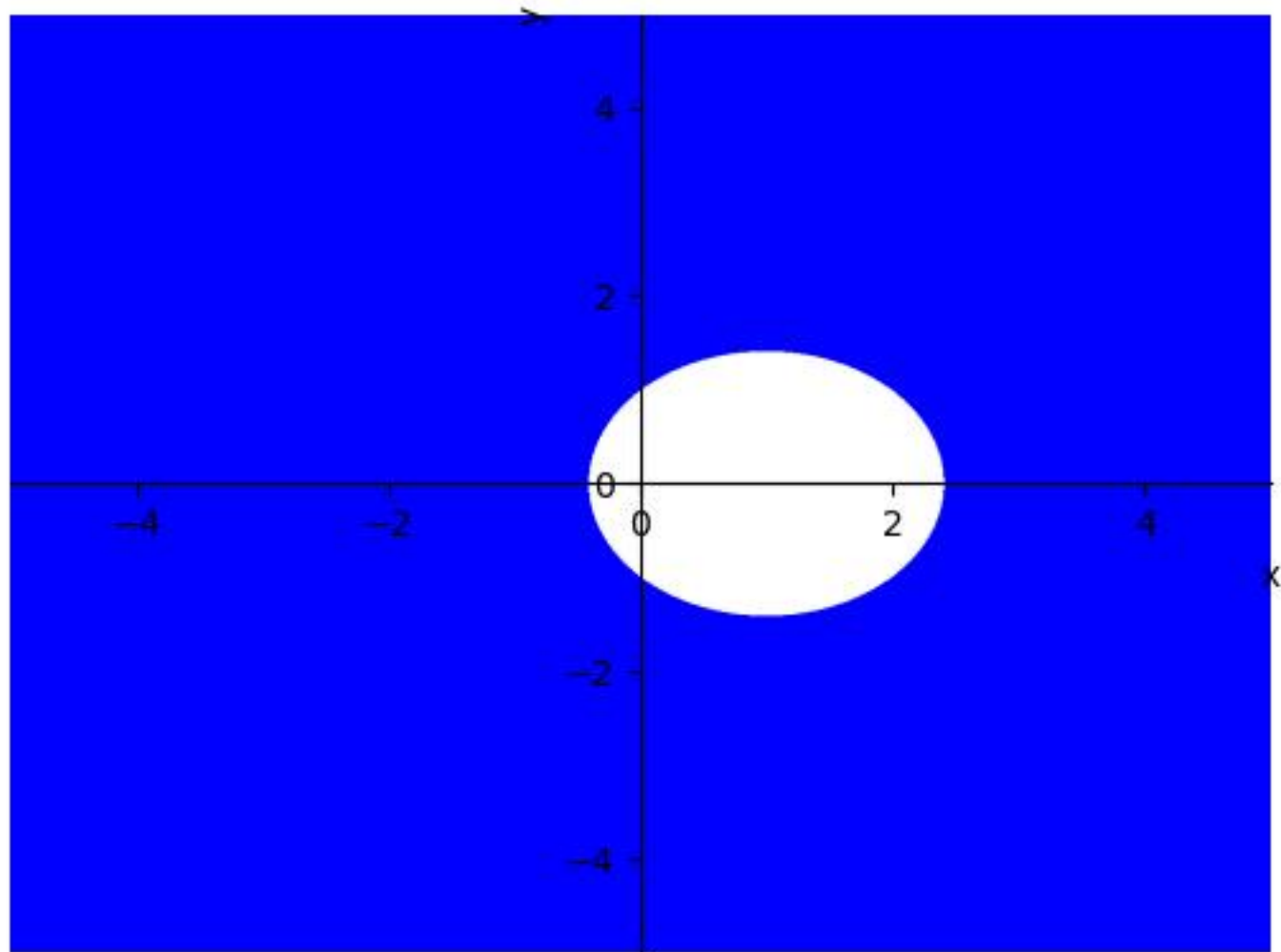


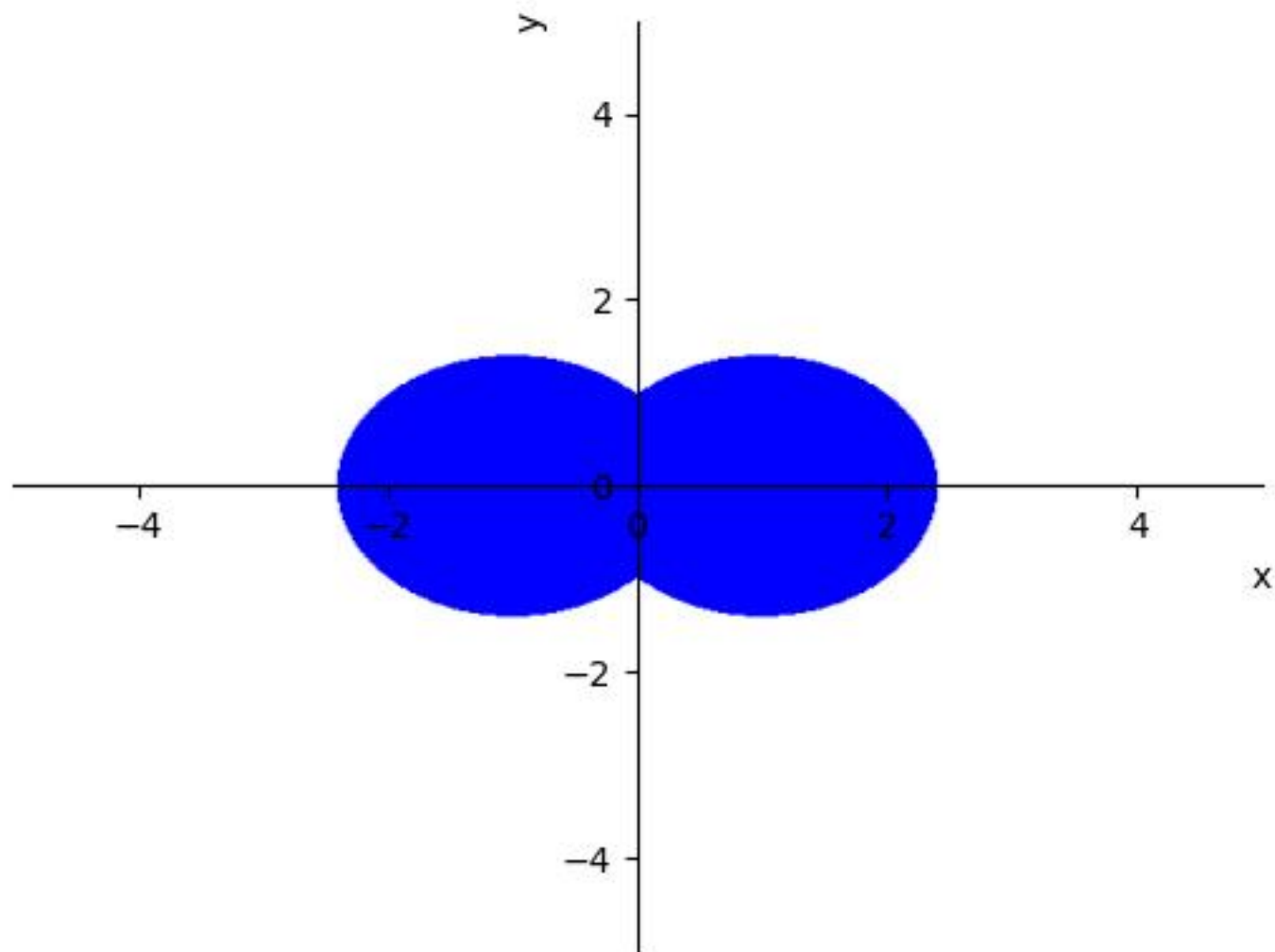


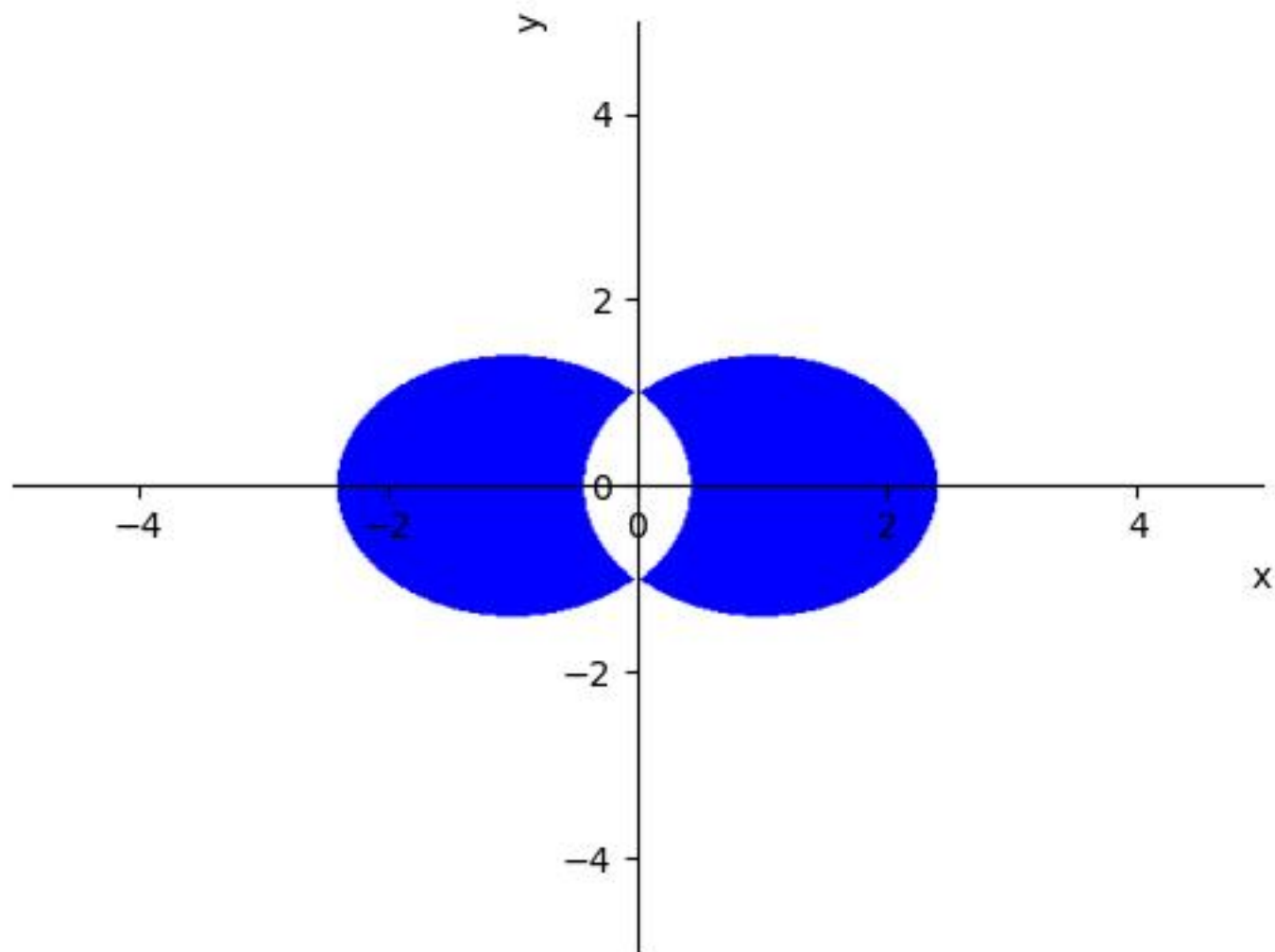
























































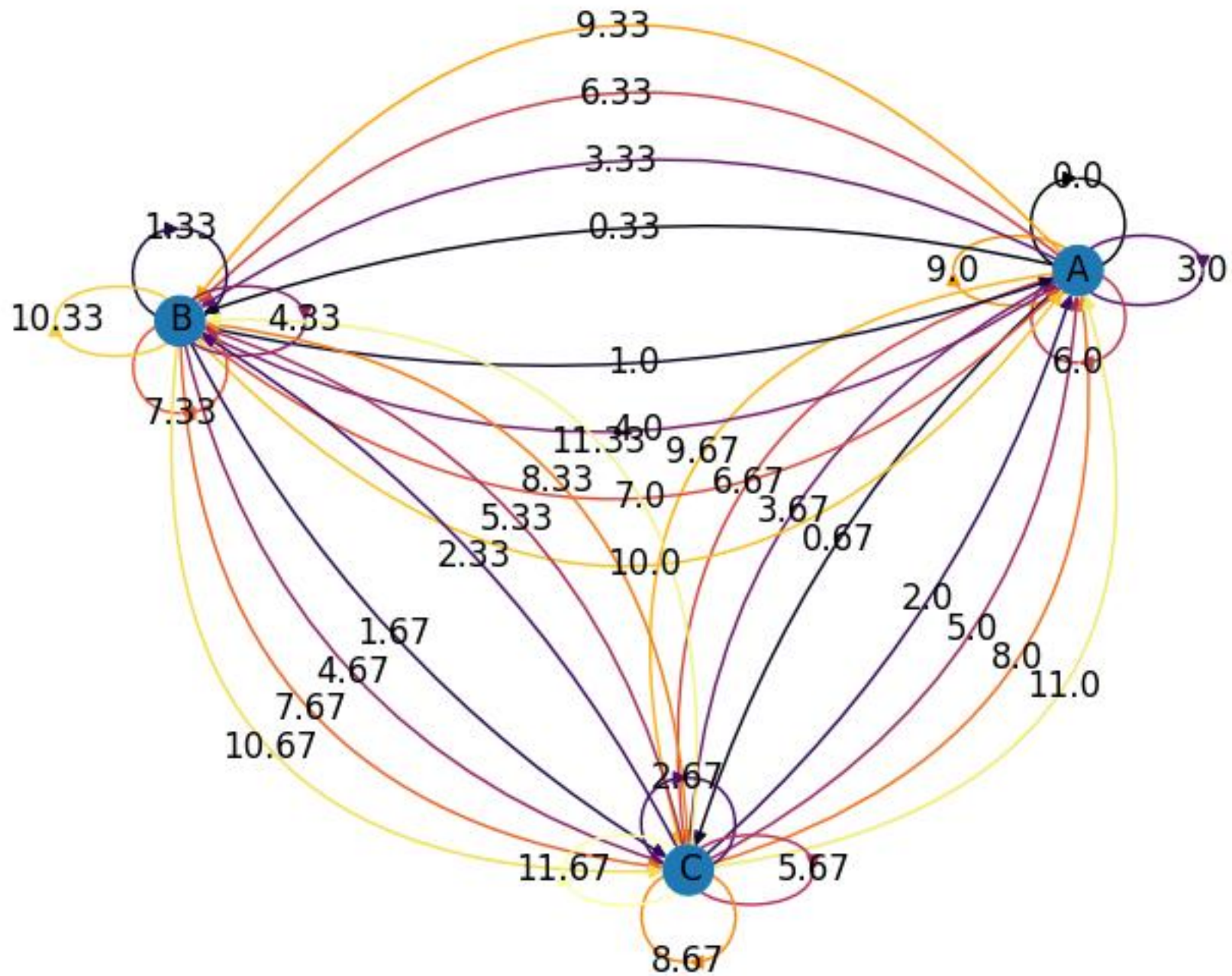


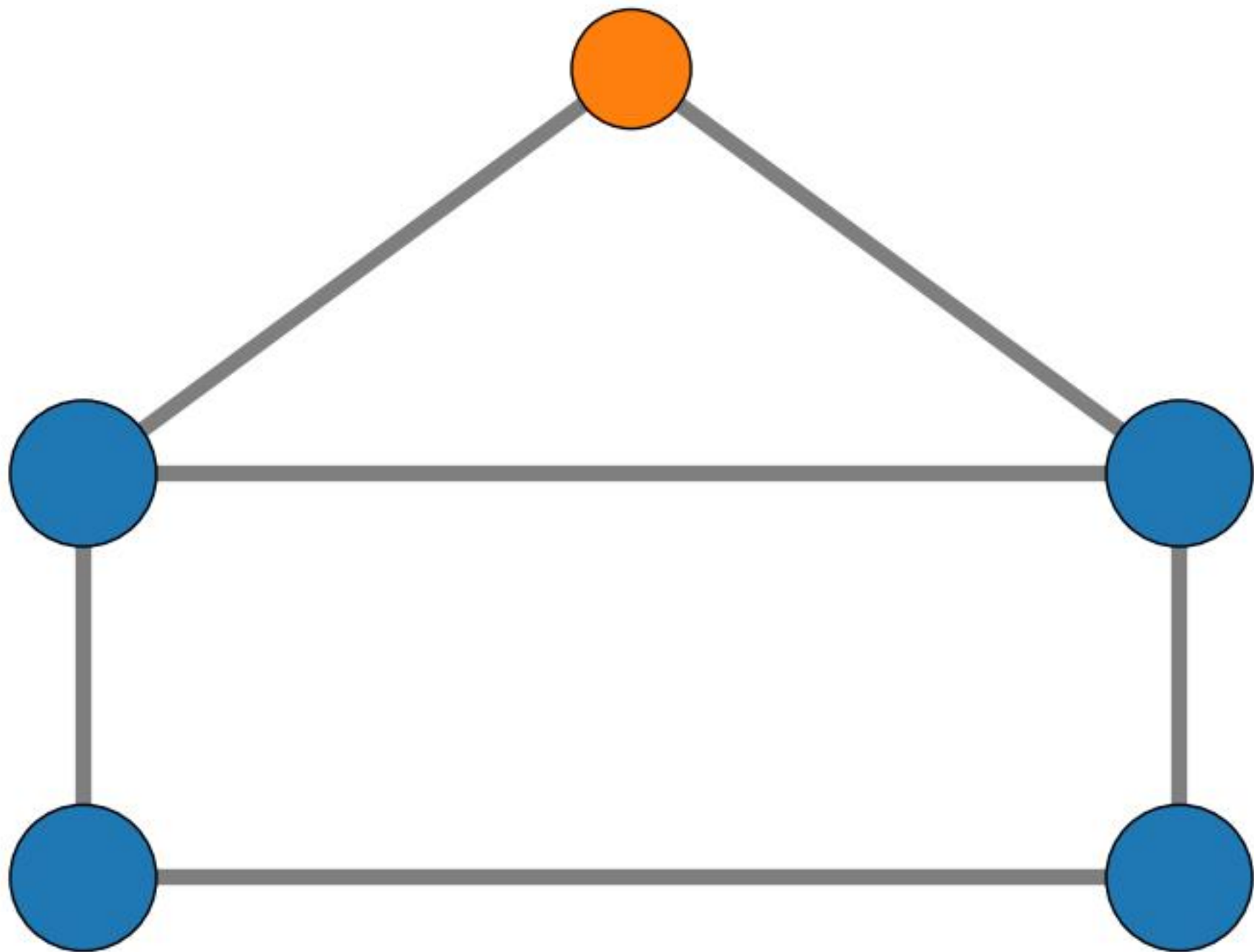


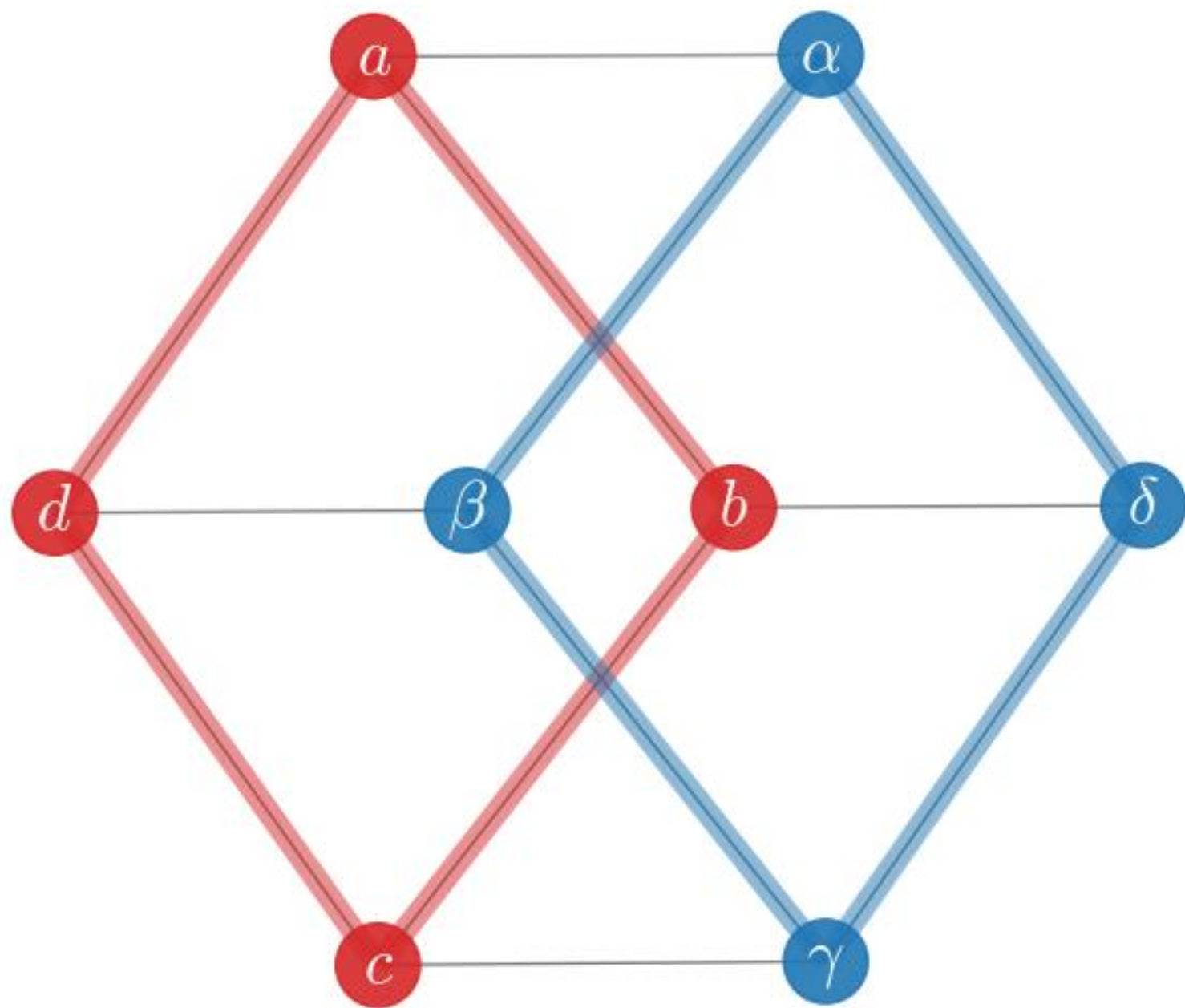


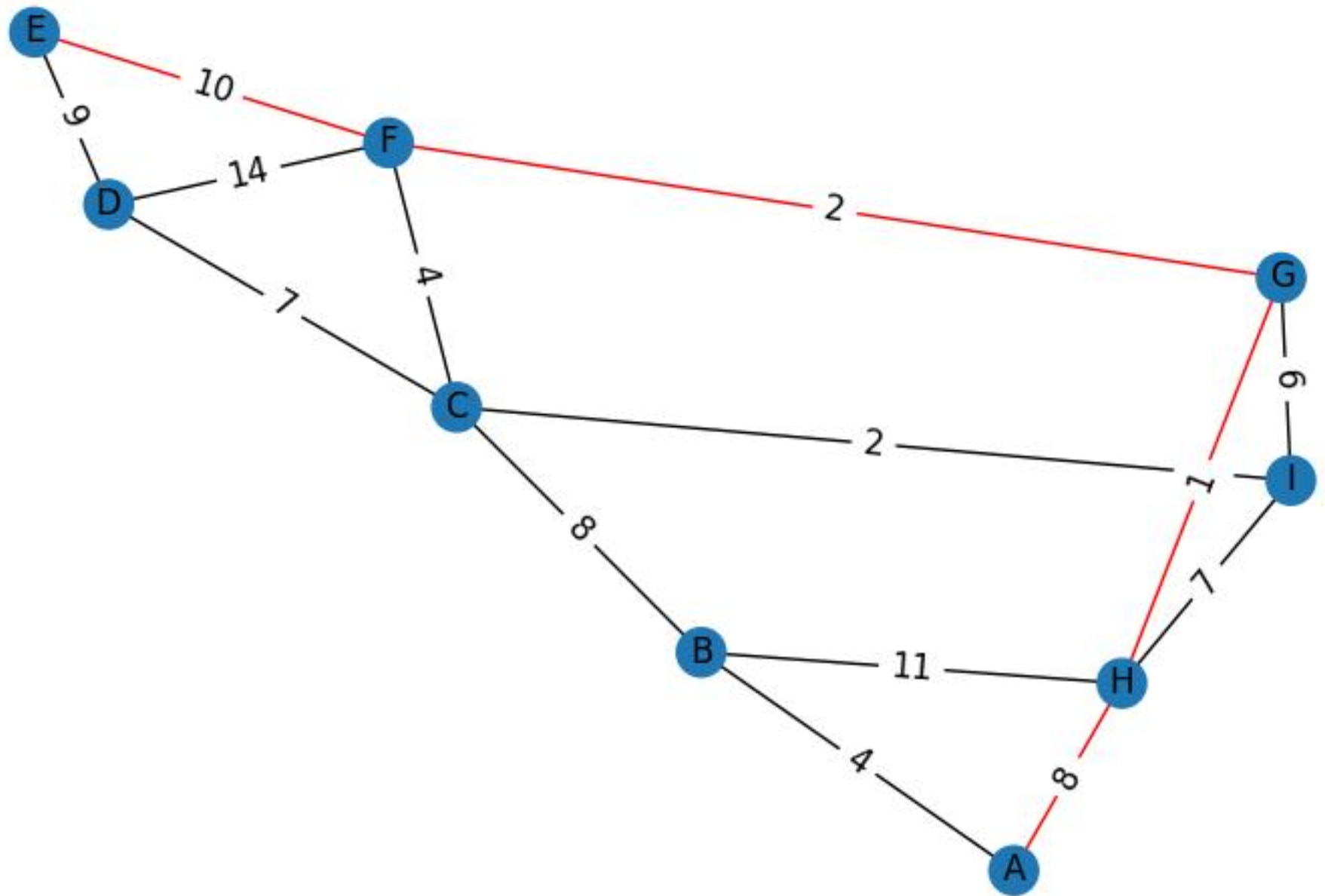


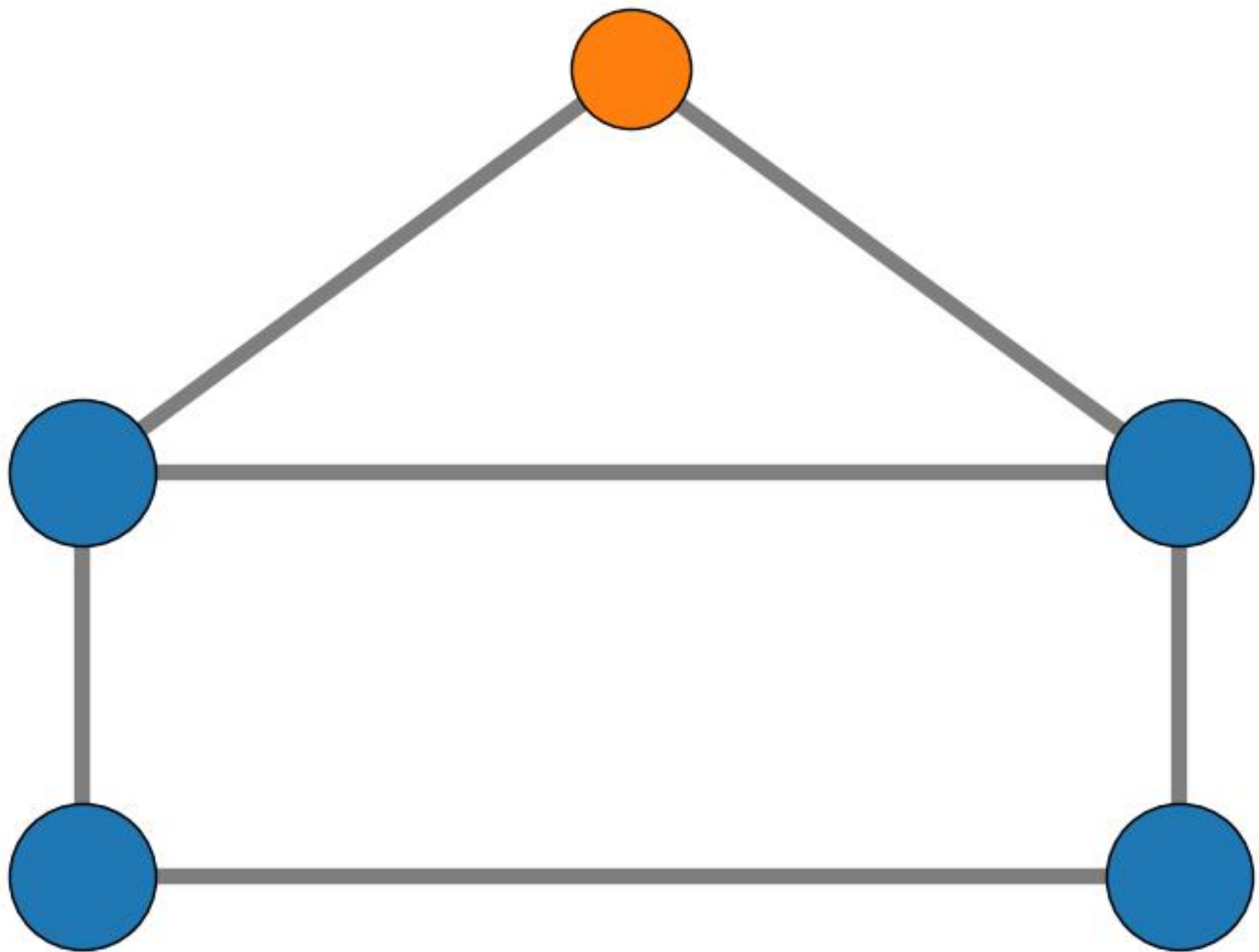








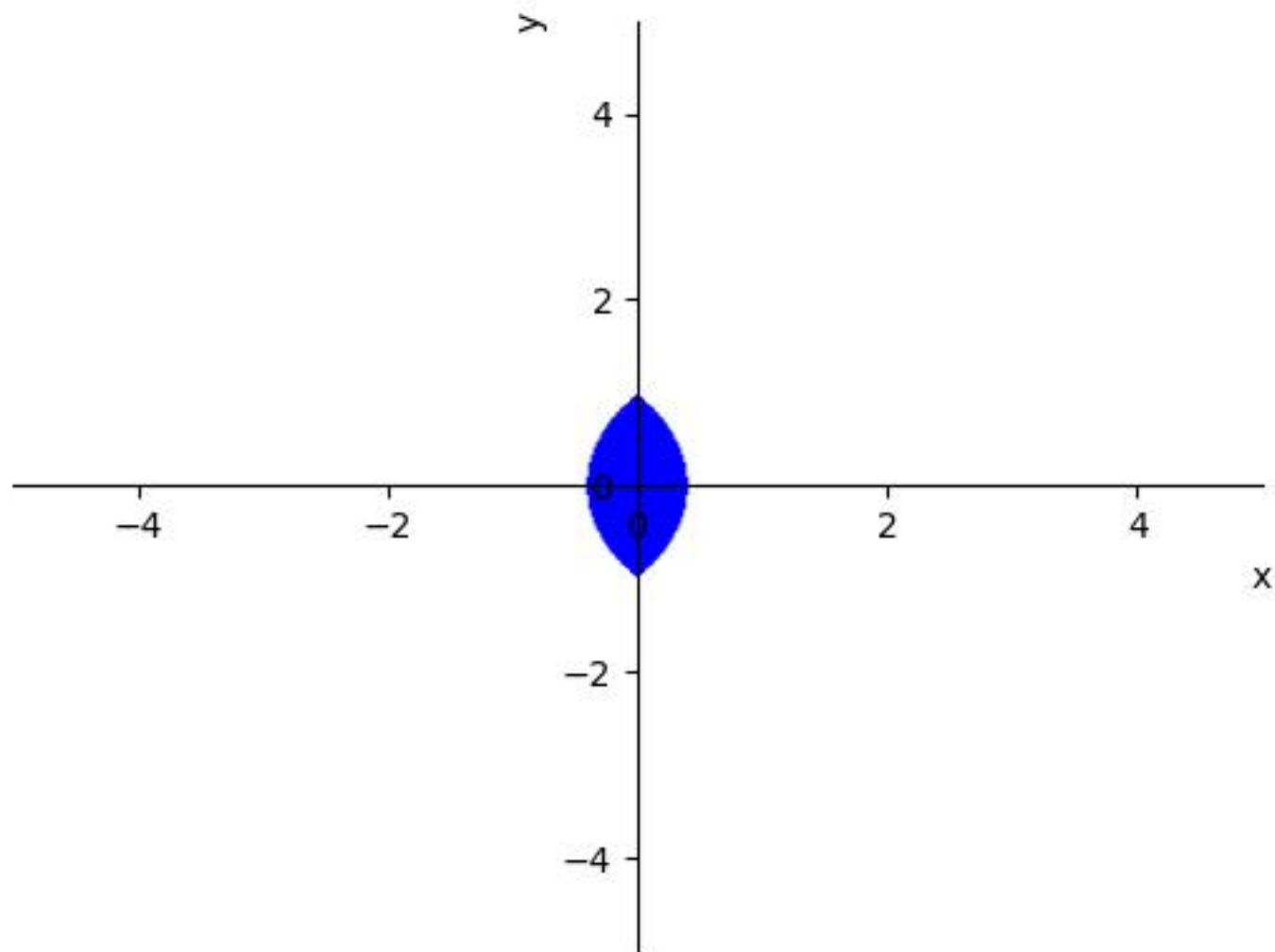


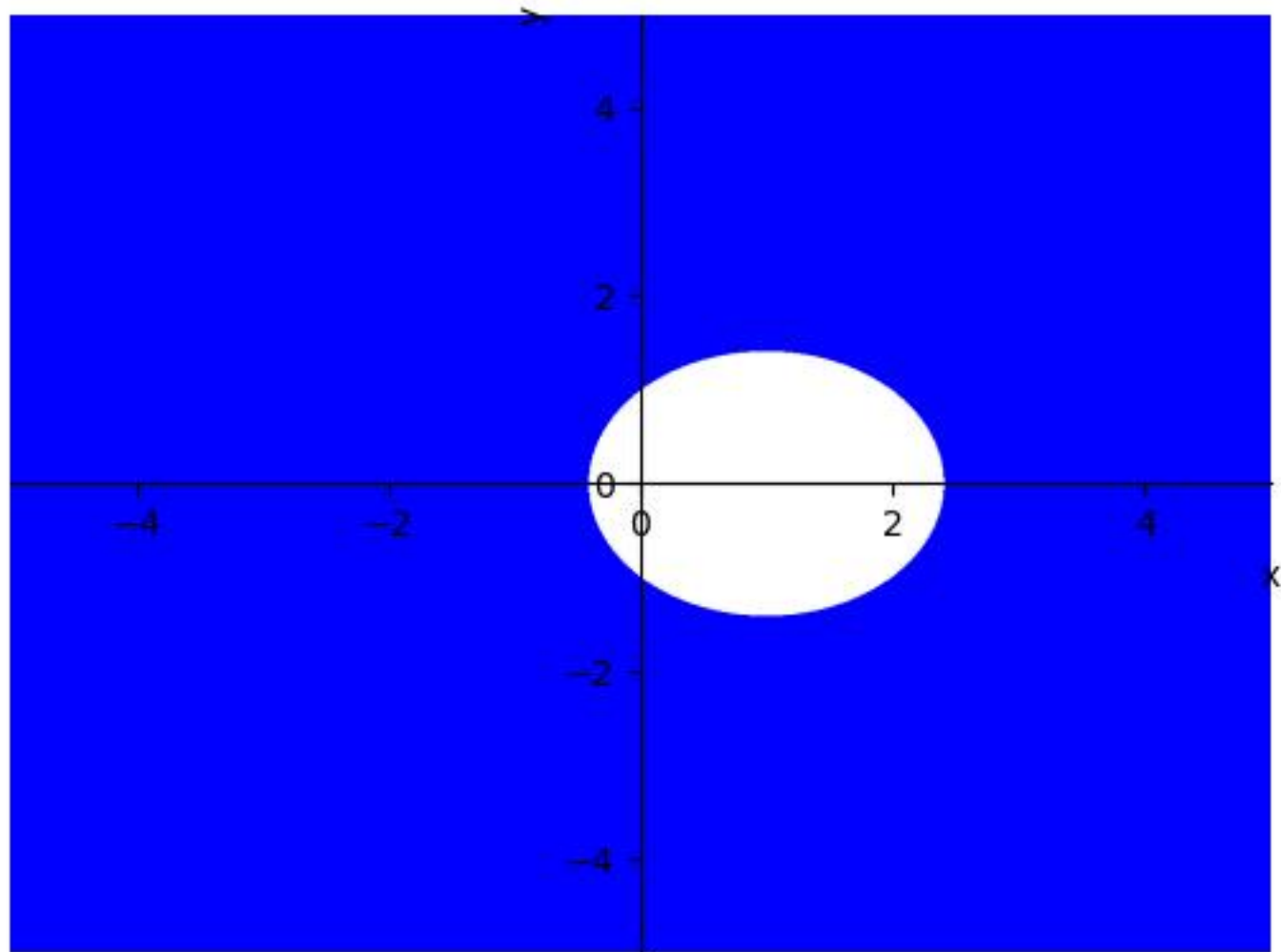


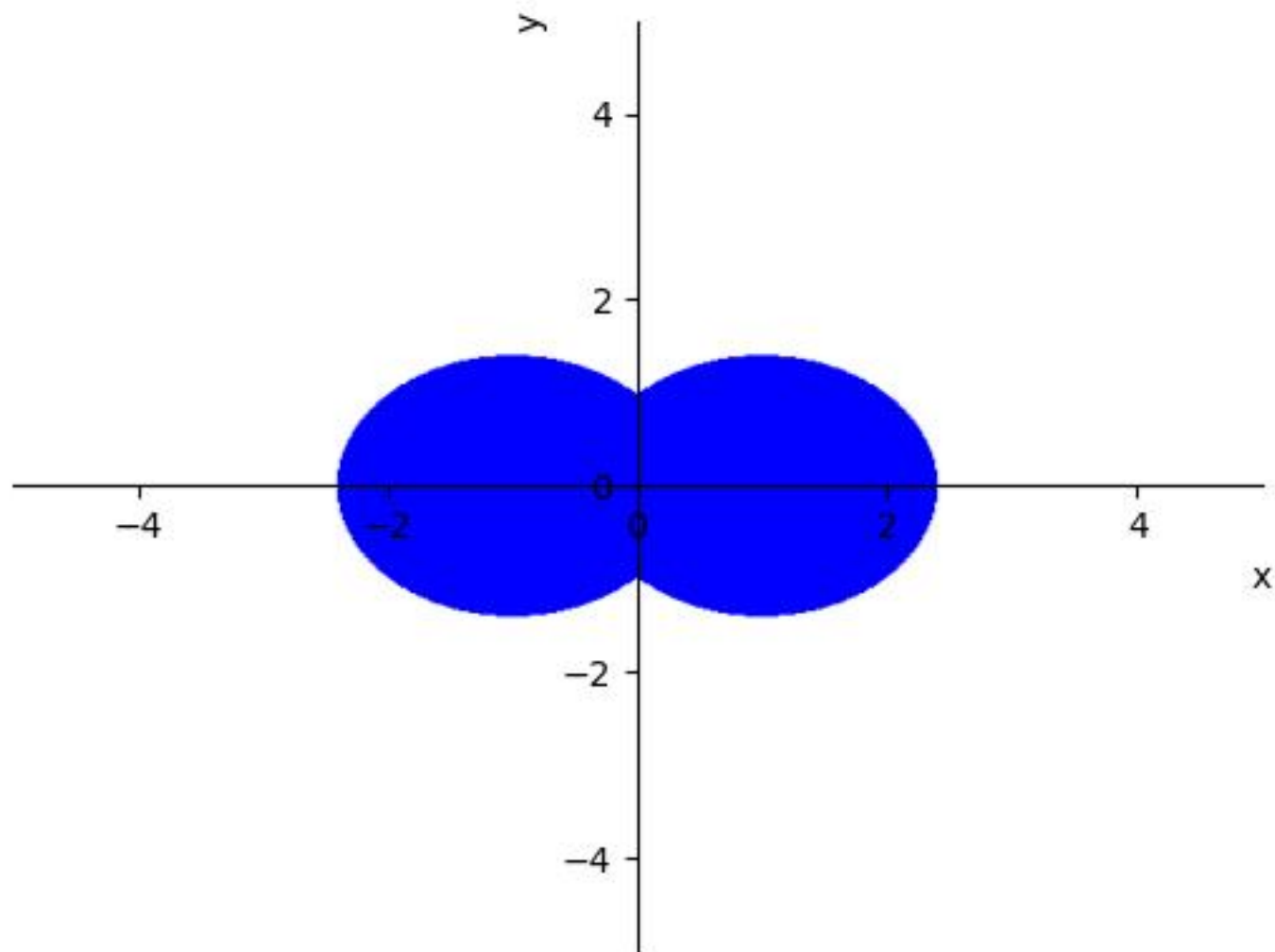


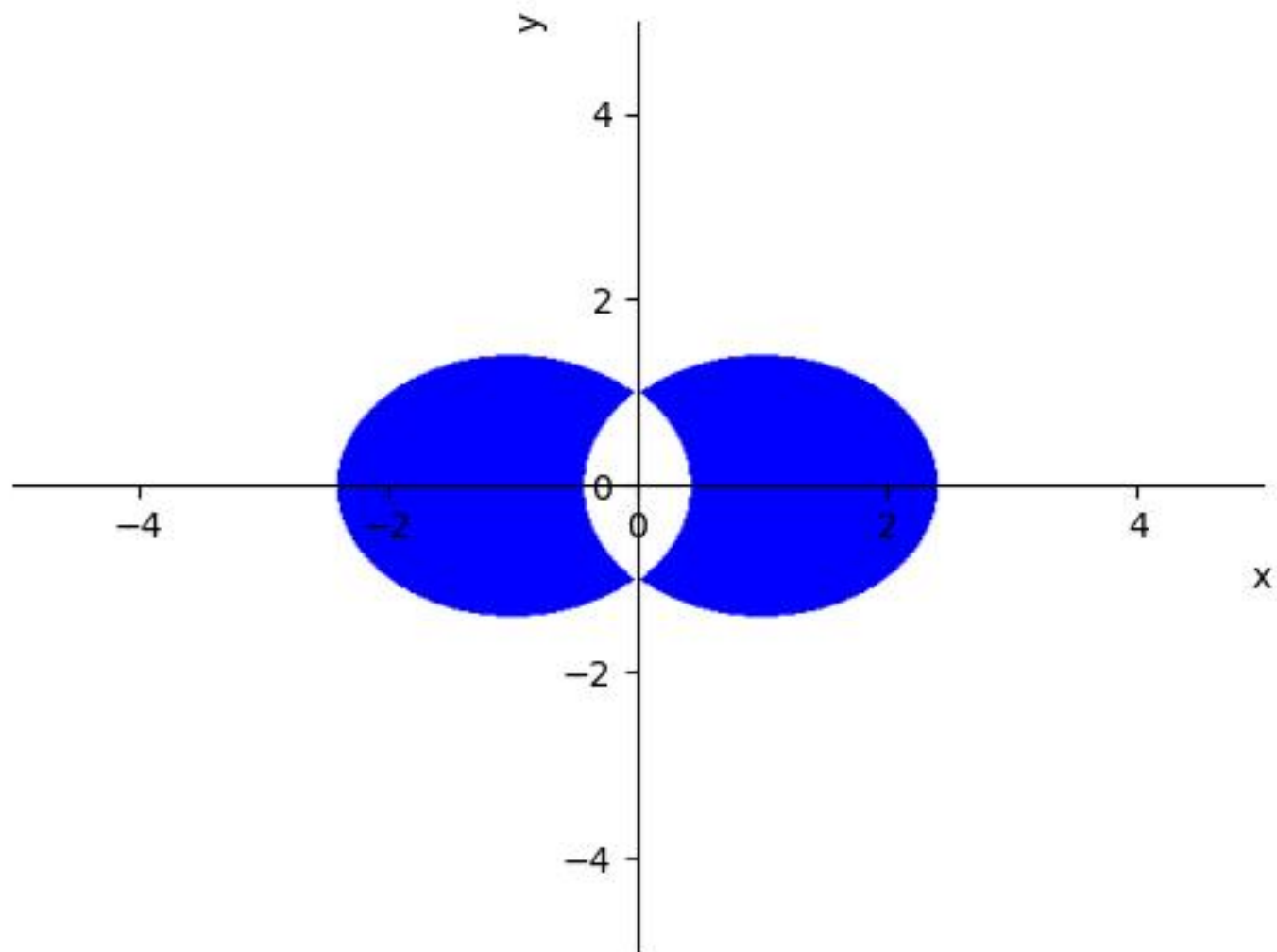










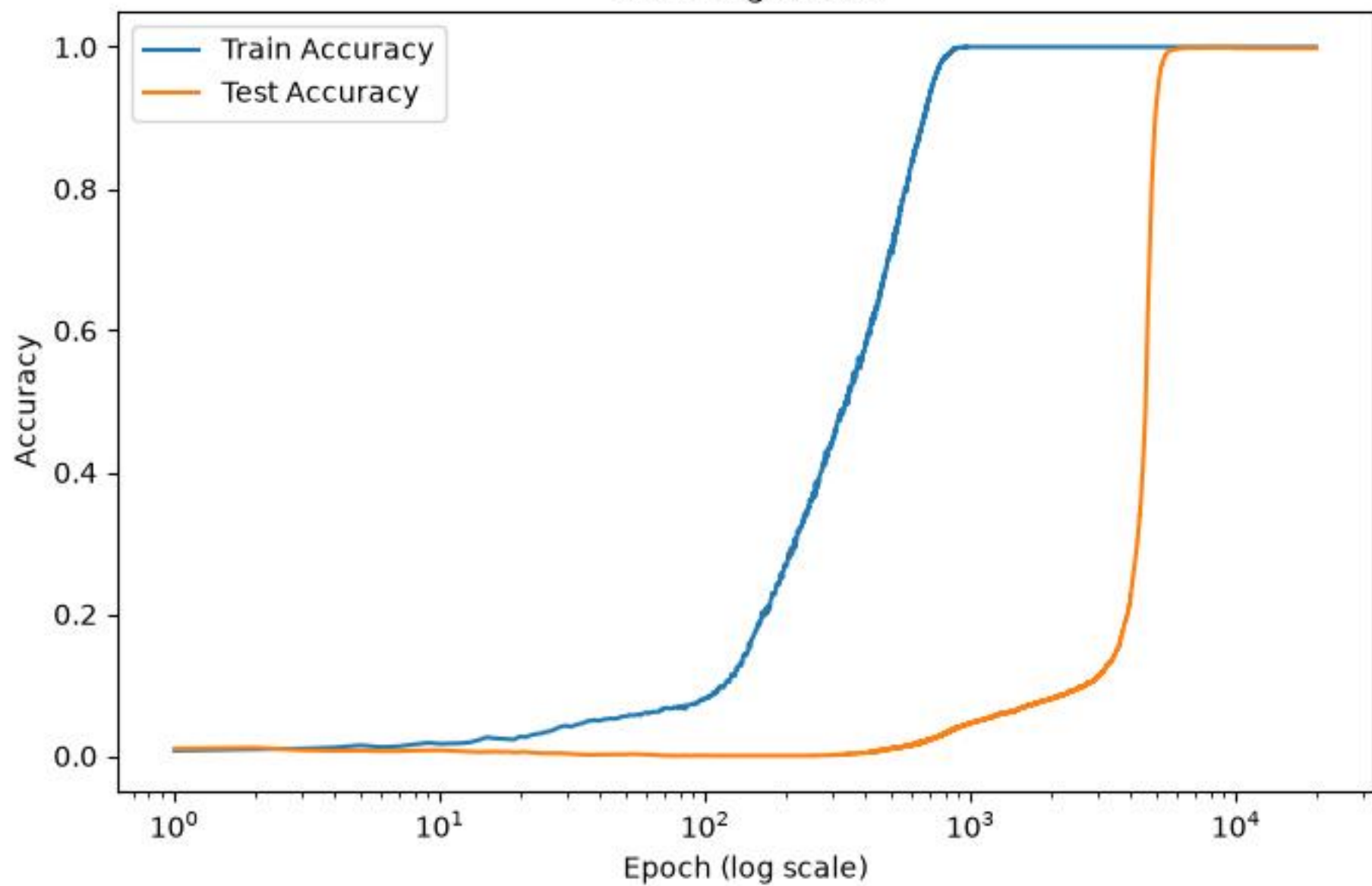


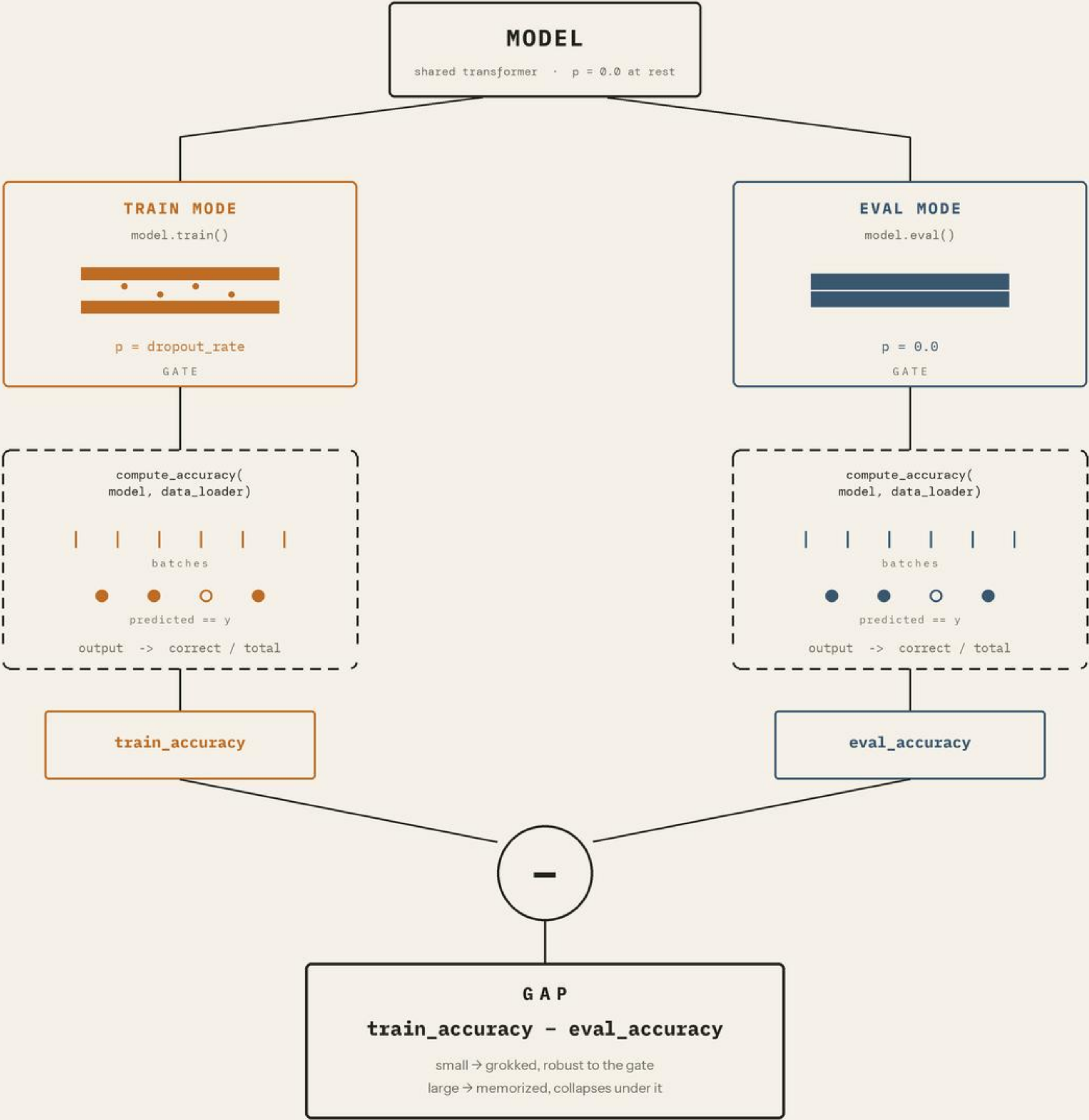




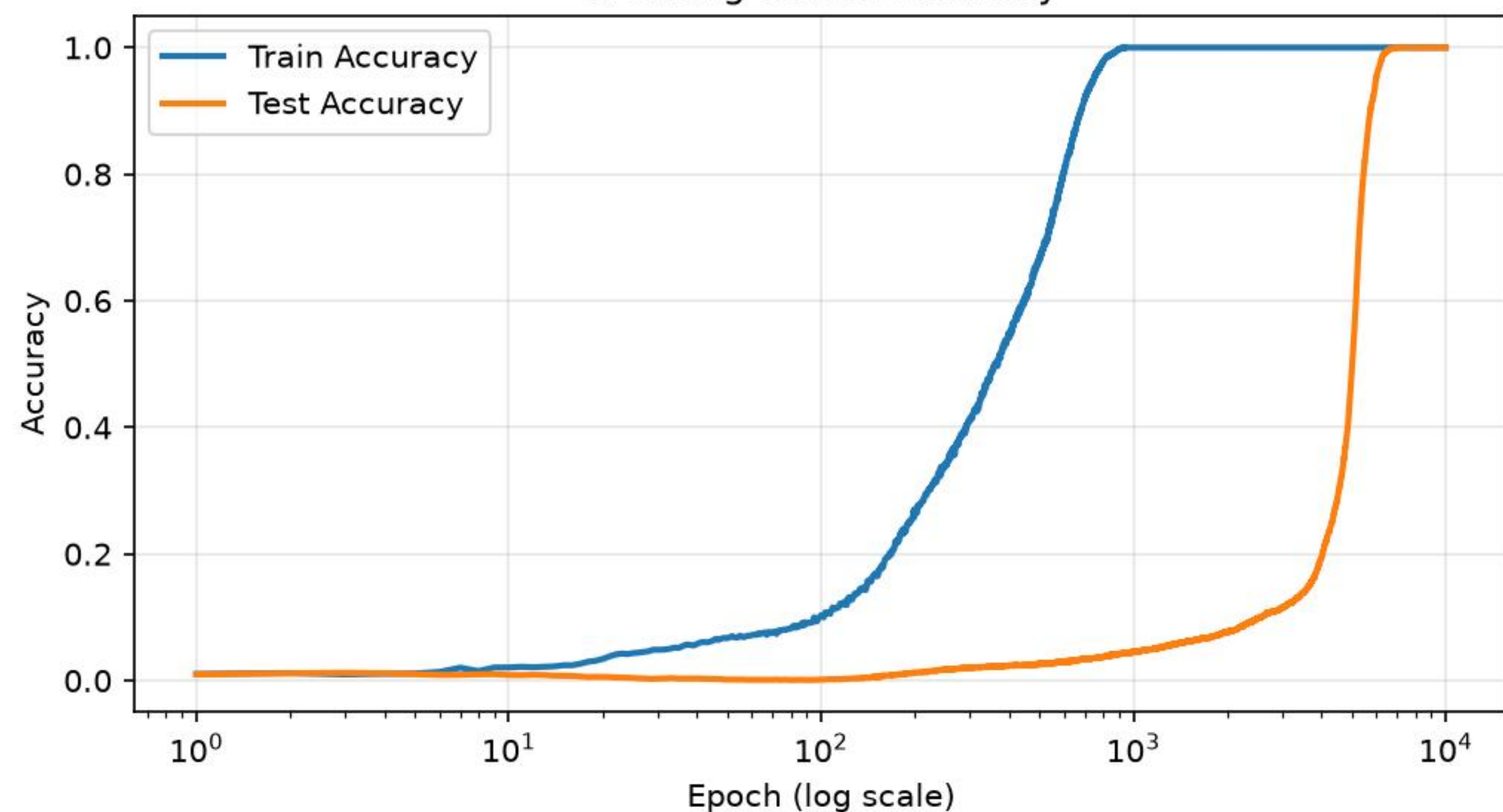


Grokking Curve

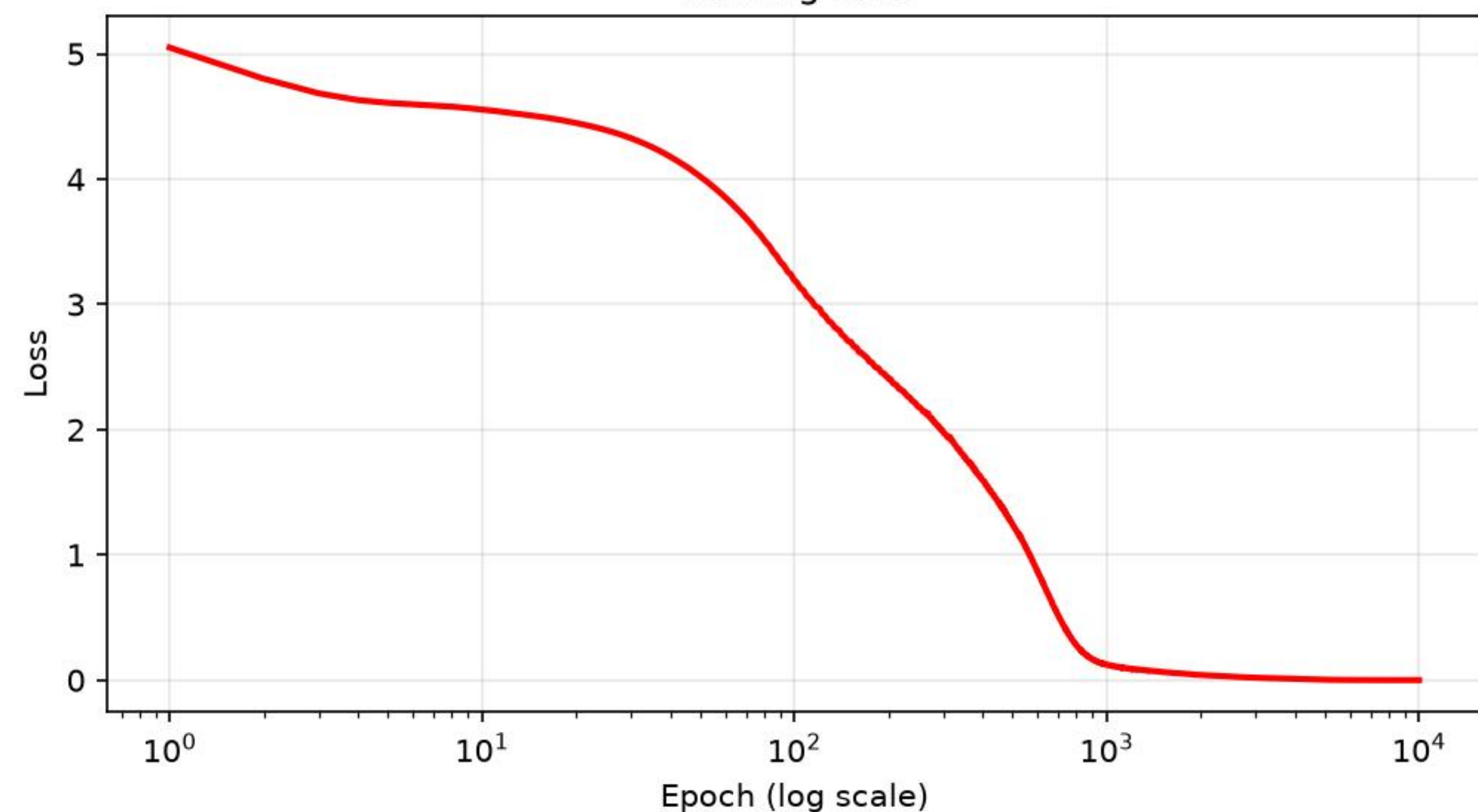




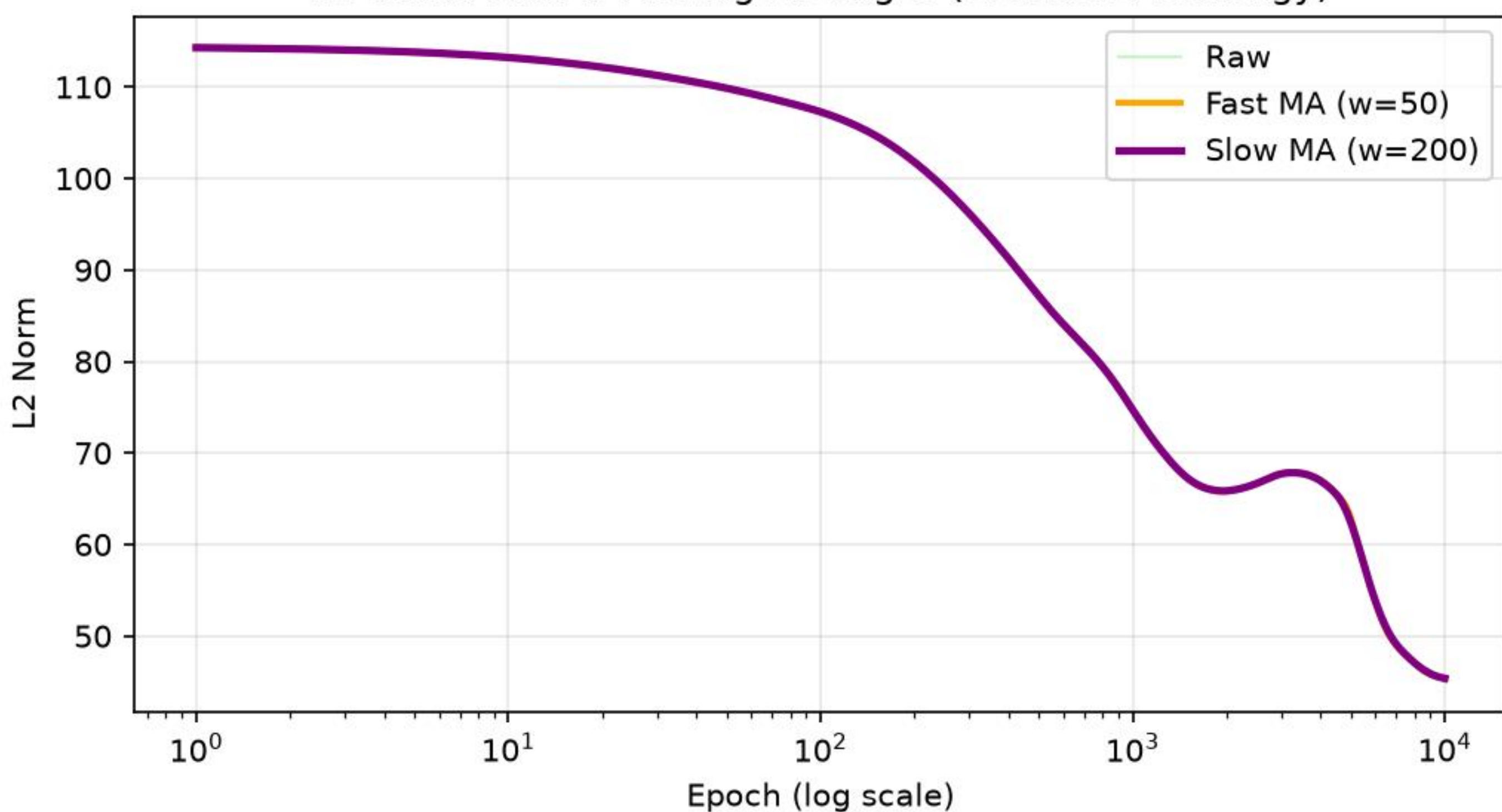
Grokking Curve: Accuracy



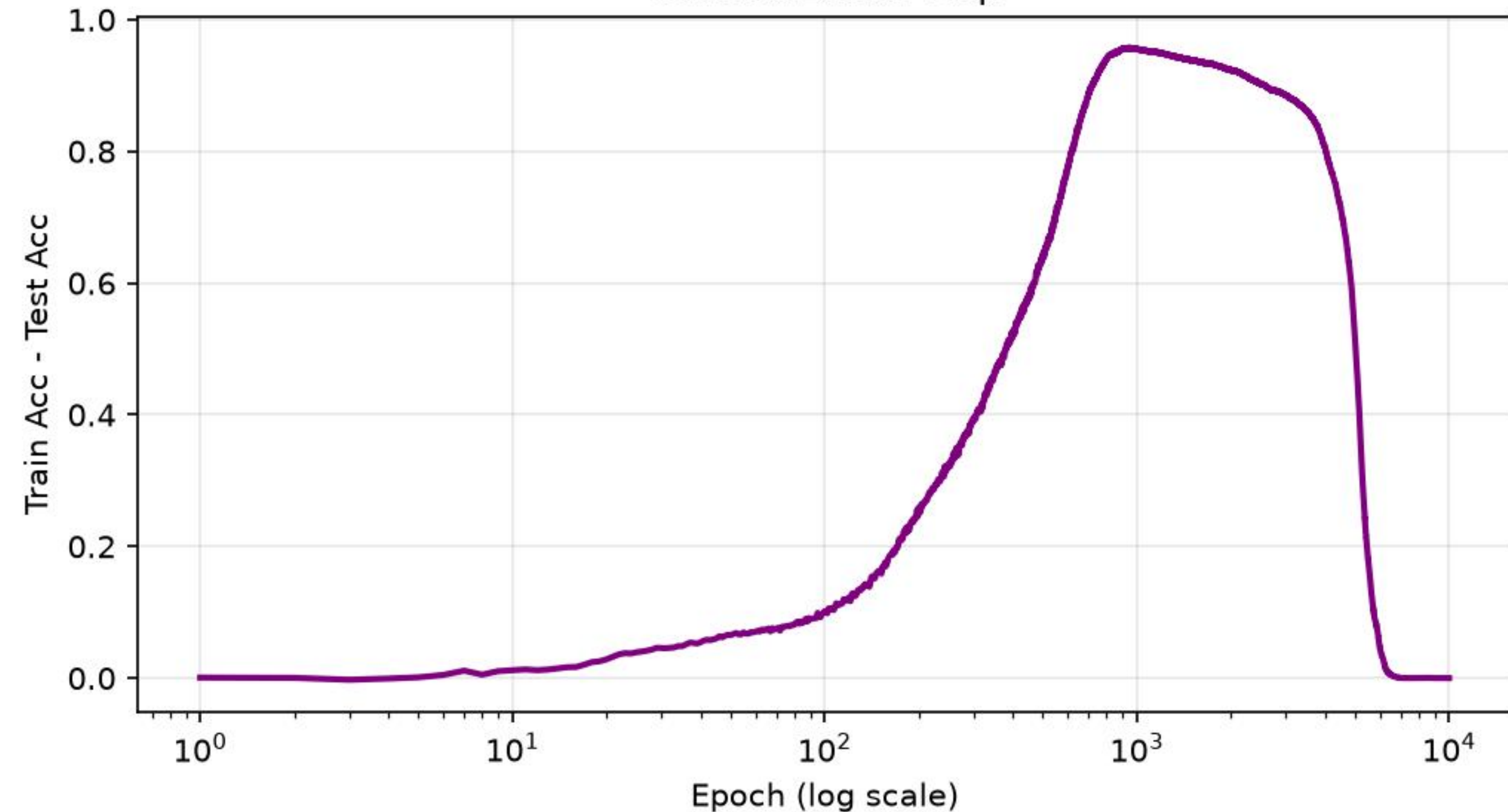
Training Loss



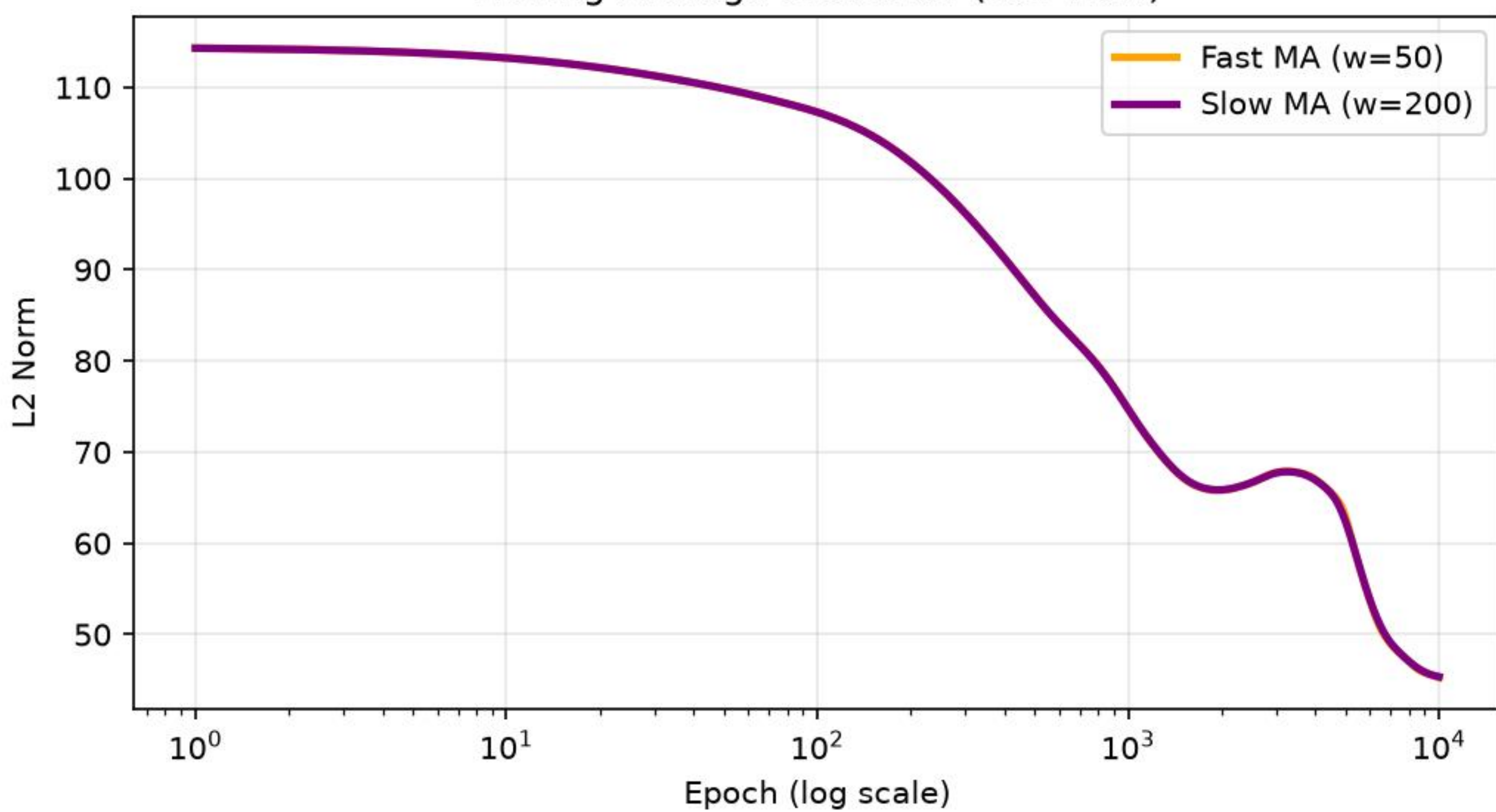
L2 Norm: Raw + Moving Averages (Crossover Strategy)



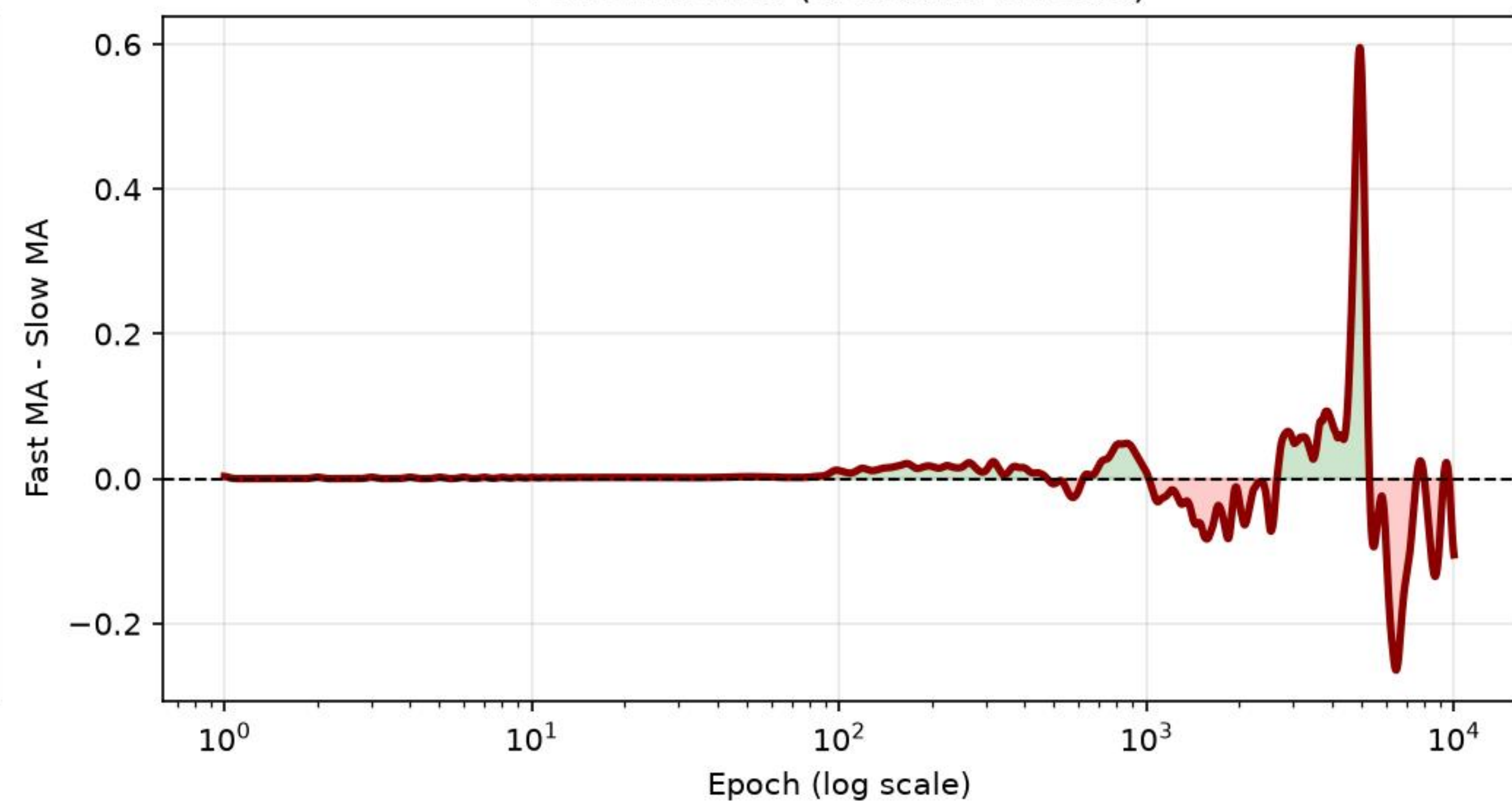
Generalization Gap



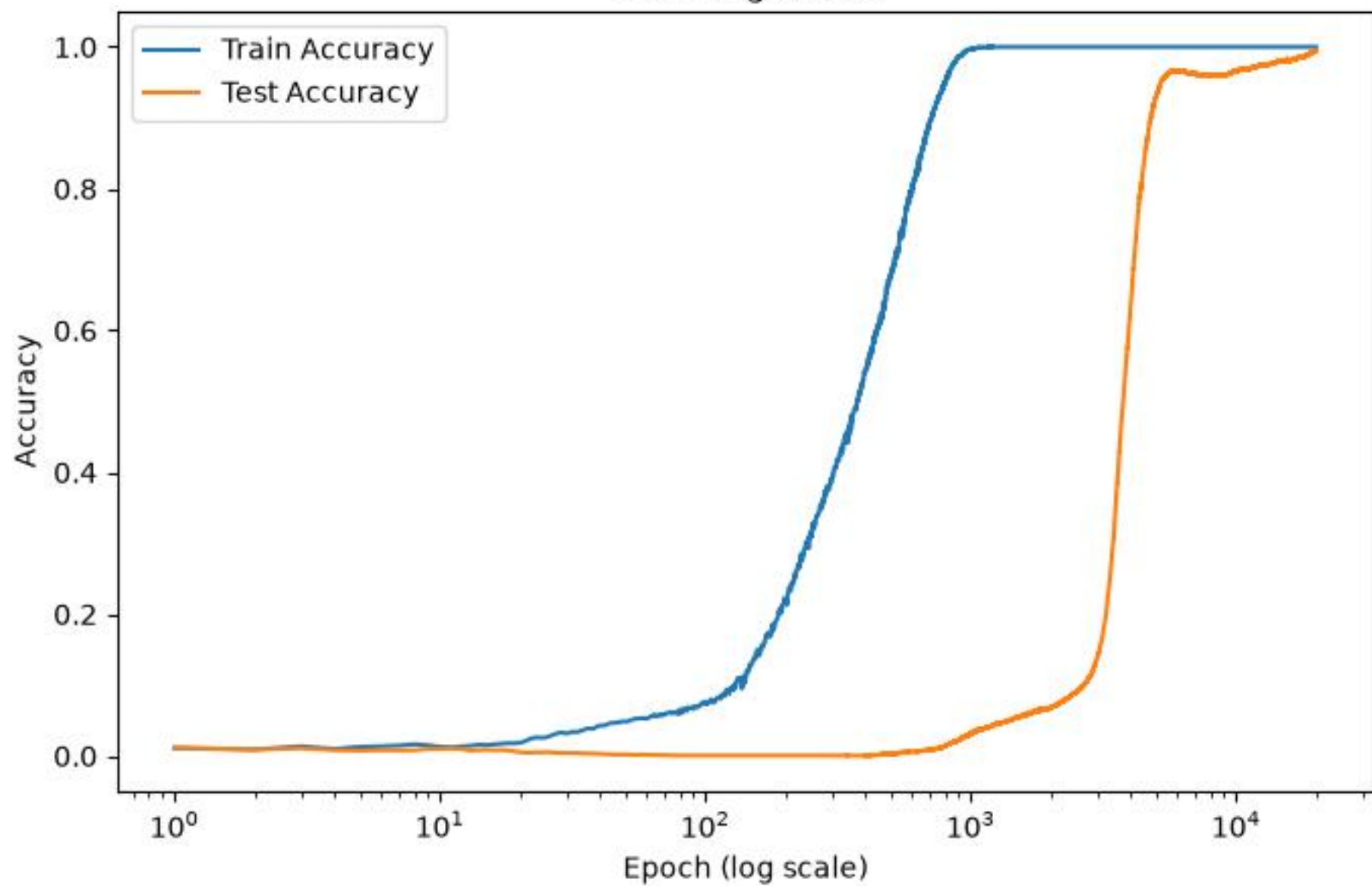
Moving Average Crossover (Full View)



MA Difference (Crossover at Zero)



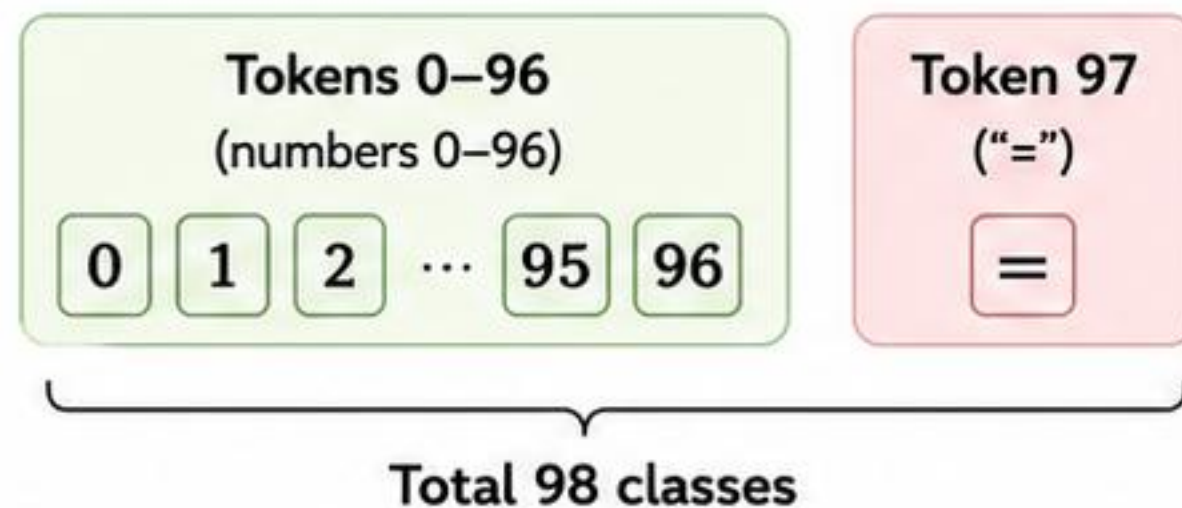
Grokking Curve



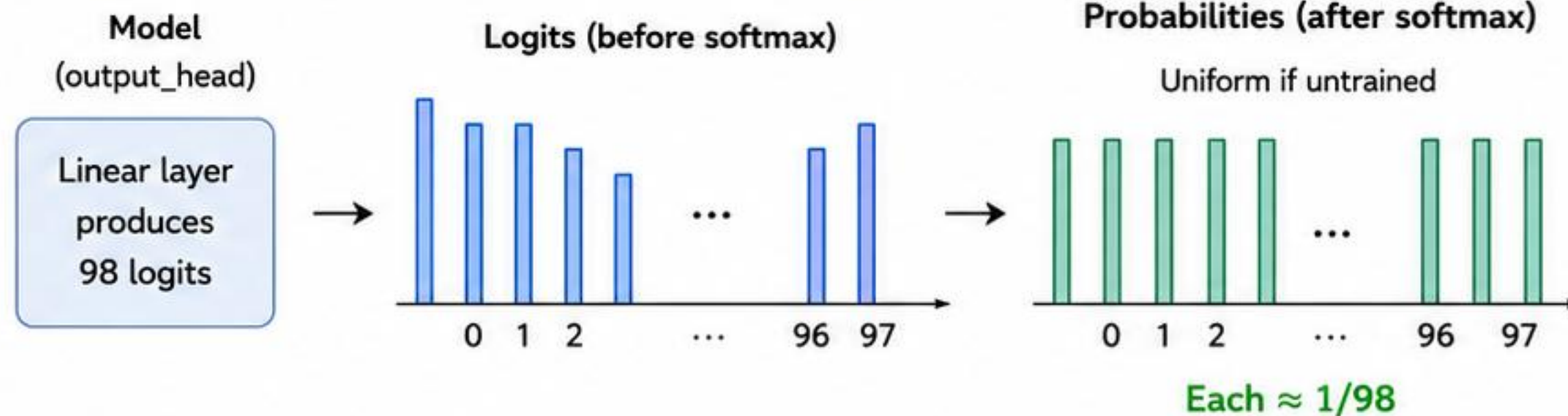
Why Initial Loss $\approx \ln(98) \approx 4.585$?

1. Setup: 98 possible tokens (classes)

vocab_size = 98



2. Model output: one logit per class



Interpretation

The model is randomly guessing among 98 options. So every class gets the same probability.

3. Cross-entropy loss (what PyTorch computes)

CrossEntropyLoss for one example:

$$L = -\ln(p_{\text{correct}}) \quad (\text{negative log-likelihood})$$

Where p_{correct} = probability assigned to the true (correct) class.

Untrained model (uniform over 98 classes):

$$p_{\text{correct}} = \frac{1}{98}$$

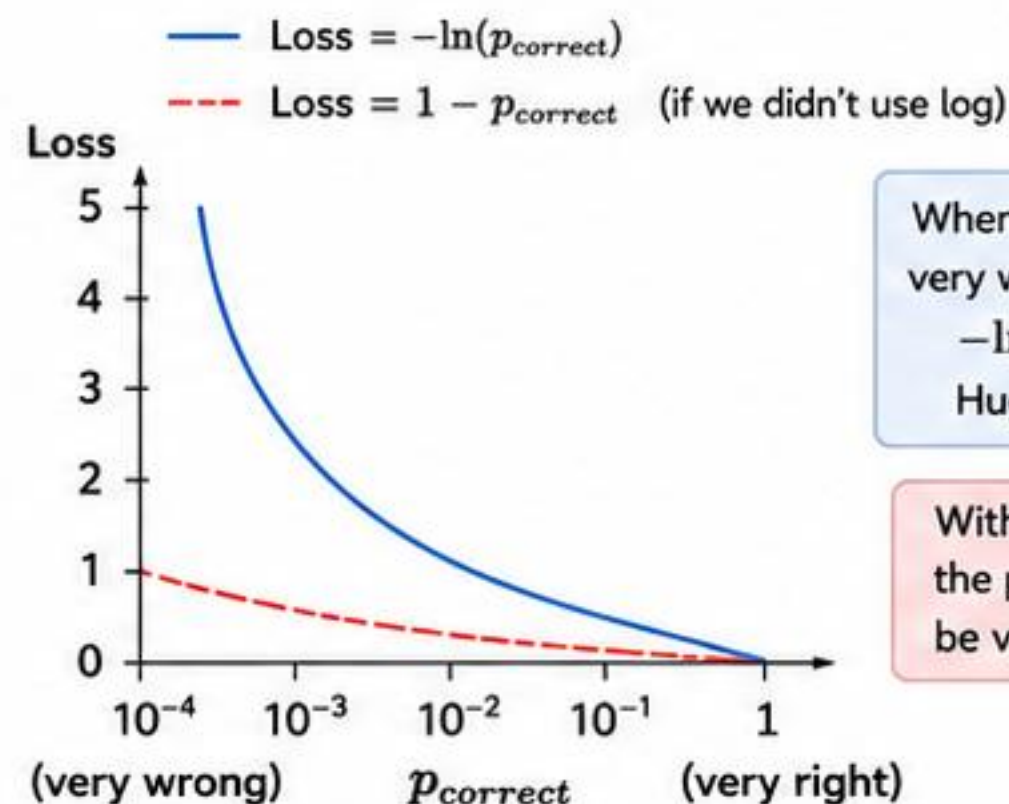
$$L = -\ln\left(\frac{1}{98}\right) = \ln(98) \approx 4.585$$

This is why initial loss ≈ 4.585 appears automatically.
It's not coded manually — it's the math.

4. Why use log? (intuition)

Cross-entropy = negative log-likelihood

It punishes confident wrong answers much more than plain probability.



5. Summary

98 classes (0-96 numbers + 97 "=")

Untrained model = uniform guess
 $p_{\text{correct}} = 1/98$

CrossEntropyLoss = $-\ln(p_{\text{correct}})$
 $= \ln(98) \approx 4.585$

Log makes the loss large when wrong, which helps the model learn faster and more reliably.

Key Takeaway

$\ln(98)$ is the natural starting point for any classification problem with 98 equally likely classes under a random (untrained) model.

Example intuition with numbers

✗ Model is very wrong/confidently wrong
 $p_{\text{correct}} = 0.0001$
 $-\ln(0.0001) = 9.210$ 😱

✗ Model is somewhat wrong
 $p_{\text{correct}} = 0.01$
 $-\ln(0.01) = 4.605$ 😞

✓ Model is somewhat right
 $p_{\text{correct}} = 0.1$
 $-\ln(0.1) = 2.303$ 😊

✓ Model is very confident & right
 $p_{\text{correct}} = 0.9$
 $-\ln(0.9) = 0.105$ 😄

PyTorch Training Loop Explained (Step by Step)

You're training a Transformer to predict $(a + b) \% 97$ at the "=" position.

A. IMPORTS (lines 1–5)

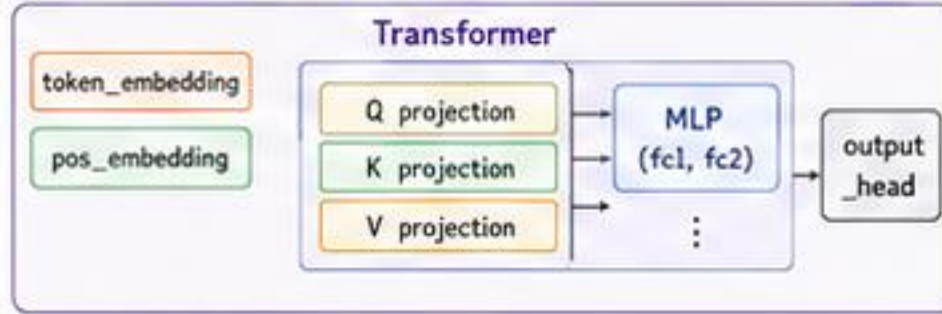
```
1. import torch.nn as nn
   # loss functions
2. from torch.optim import Adam
   # optimizer (updates weights)
3. from utils.data_utils import get_dataloaders
   # returns (train_loader, test_loader)
4. from model import Transformer
   # your Transformer model
5. import torch
```

B. SETUP (lines 7–10)

```
7. data_loader = get_dataloaders(97, batch_size=32)
   # returns (train_loader, test_loader)
   # each batch contains 32 sequences
8. model = Transformer(vocab_size=98, d_model=128)
   # 98 = 97 numbers (0–96) + "=" token
9. optimizer = Adam(model.parameters(), lr=0.001)
   # Adam will update every learnable weight
   # lr = learning rate (step size)
10. cross_entropy_loss = nn.CrossEntropyLoss()
    # measures how wrong the predicted
    # distribution (logits) is vs true class
```

WHAT ARE `model.parameters()`?

It returns an iterator over ALL learnable tensors (weights and biases) in the model.

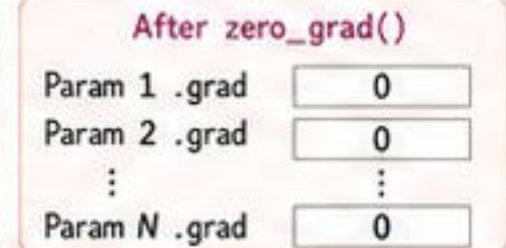
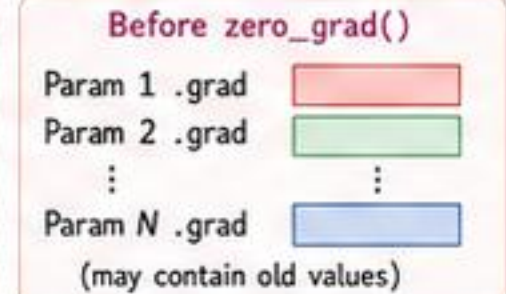


Adam keeps track of each of these parameters and updates them using gradients.

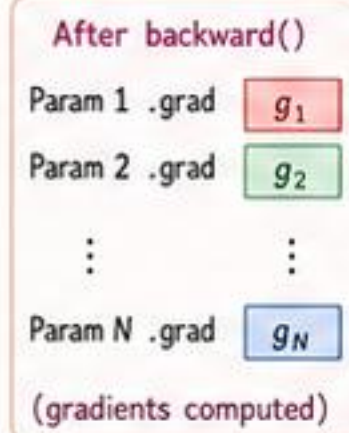
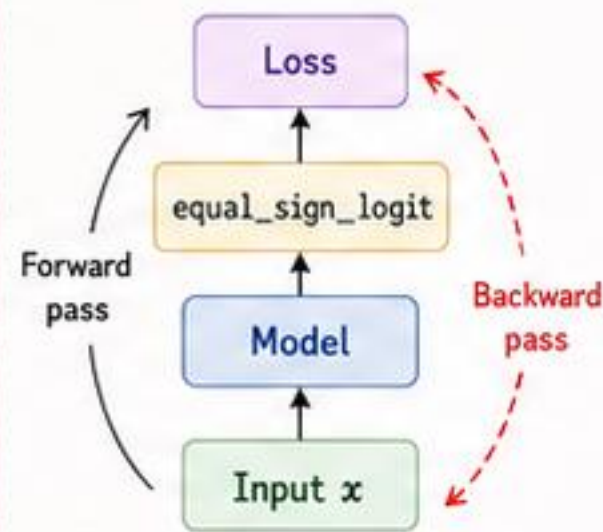
C. TRAINING LOOP (lines 13–24)

```
13. for epoch in range(100):           # 100 full passes over the dataset
14.     for x, y in data_loader[0]:      # iterate over train_loader
        # x: input sequences -> shape (batch, 3)
        # y: true targets (a+b)%97 -> shape (batch,)
17.     logit = model.forward(x)
        # logits shape: (batch, seq_len=3, vocab_size=98)
18.     equal_sign_logit = logit[:, 2, :]
        # take only position 2 (the "=" position)
        # shape: (batch, 98)
19.     loss = cross_entropy_loss(equal_sign_logit, y)
        # compares predicted distribution vs true class
        # returns a scalar loss
```

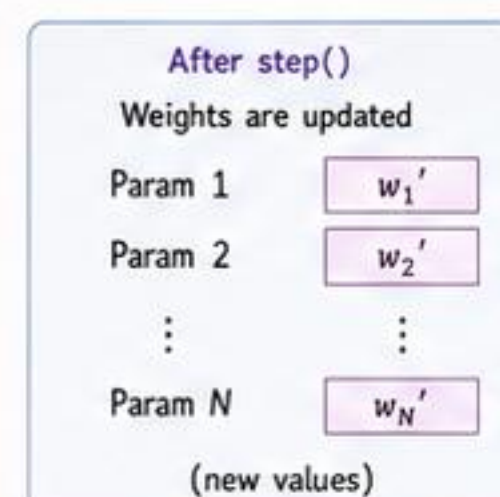
1 20. `optimizer.zero_grad()`
clear old gradients (very important!)



2 21. `loss.backward()`
backprop: compute gradient of loss w.r.t. every weight



3 22. `optimizer.step()`
Adam updates weights using those gradients



```
23. (loop continues for next batch)
24. print(f"Epoch: {epoch}, Loss: {loss.item():.4f}")
    # prints the loss of the LAST batch in this epoch
```

WHY THIS ORDER? (zero_grad → backward → step)

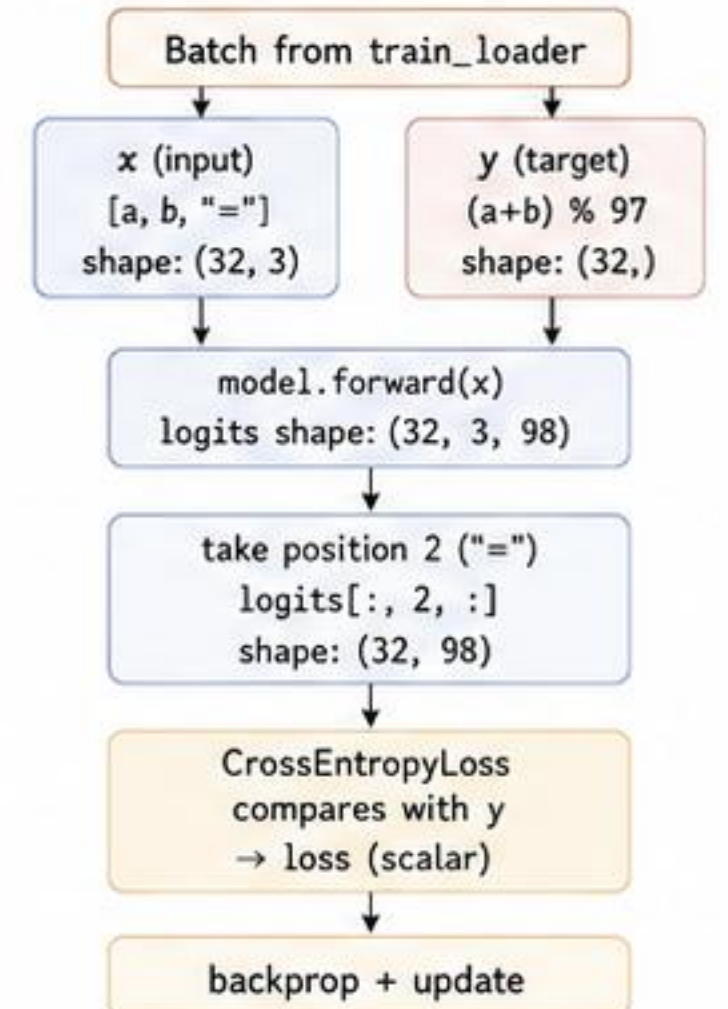
1. `zero_grad()`
Clears old gradients. If you don't, gradients will accumulate and make updates wrong.

2. `backward()`
Computes gradients of current loss w.r.t. every parameter.

3. `step()`
Uses gradients to update weights (Adam uses them with $lr=0.001$).

Repeat for next batch and next epoch until model learns!

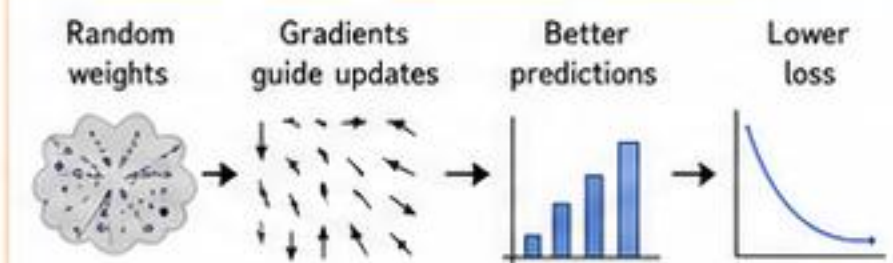
D. DATA FLOW (one batch)



E. KEY POINTS

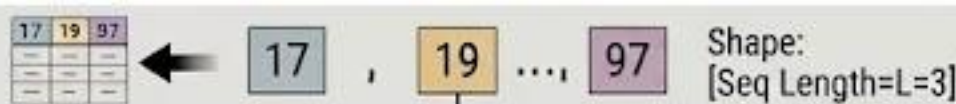
- We predict only at the "=" position (index 2).
- `CrossEntropyLoss` expects raw logits (not softmax).
- Adam uses gradients + lr to update every parameter.
- `print` shows loss from the last batch of the epoch, not the average.

F. WHAT HAPPENS DURING TRAINING?



Model gradually learns to output the correct number at the "=" position.

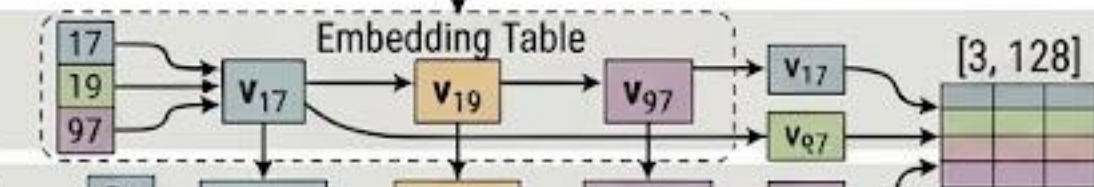
Dataset from R Input IDs Raw: [17, 19, 97]



Raw data (like word IDs) are numerical tokens for the model.

2. Token Embedding (nn.Embedding):

`nn.Embedding(vocab_size=98, d_model=128)`

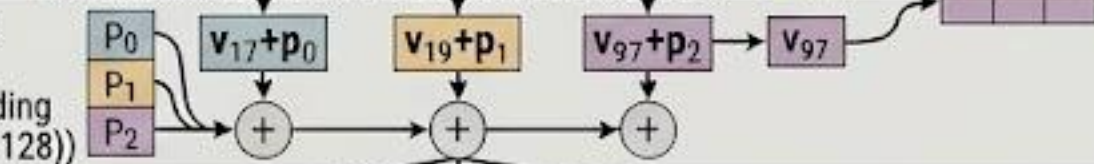


nn.Embedding(vocab): Converts each token ID into a specific 128-dimensional vector.

3. Positional Embedding

4. + Addition

Position IDs
[0, 1, 2]
position, embedding
(`nn.Embedding(3, 128)`)



Positional Embedding + Addition: Adds a vector to provide context of token order (e.g., first, second).

5. Query, Key, Value Creation



Linear Projection Layer
(W, Q) 128→128

Q [3, 128]

W_K

K [3, 128]

W_V

V [3, 128]



Example Input Sequence: [17, 19, 97]

Query (Q): What to look for? (Target: $(17+19)\%N$)

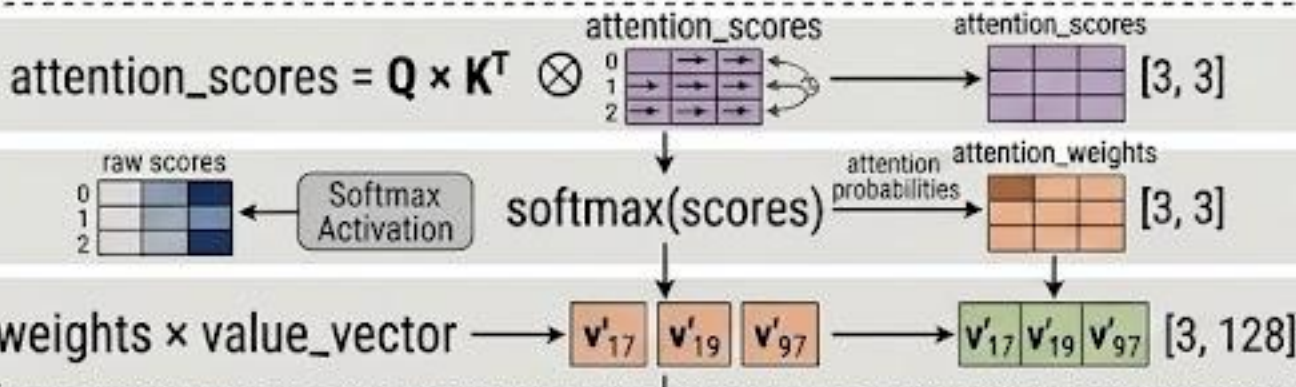
Key (K): What am I? (shows full identity)

Value (V): What will I provide?

6. Attention mechanism

7. Attention mechanism

8. $\text{attended_values} = \text{attention_weights} \times \text{value_vector}$

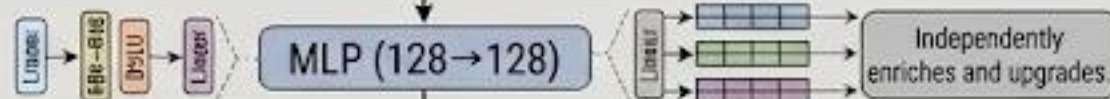


$Q \times K^T$ (Dot Product): Comparison score of each token with all others (attention region).

Softmax: Converts scores into normalized weights. (sum=1; how much attention to pay to what)

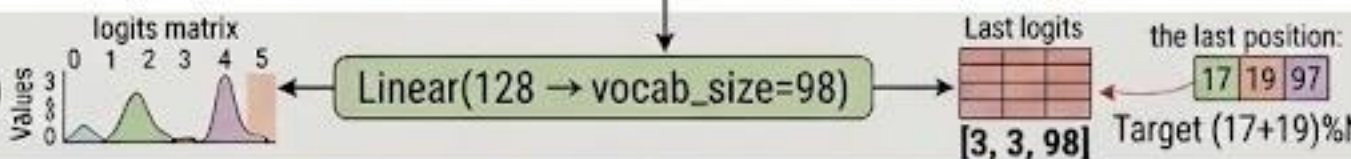
weighted value: For creating a context-aware vector (enriches the token with information from other tokens)

9. MLP (Feed-Forward Neural Network)



MLP: Modified and upgrades each token vector Independently. (To refine the mixed information)

10. Output Head (output_head)

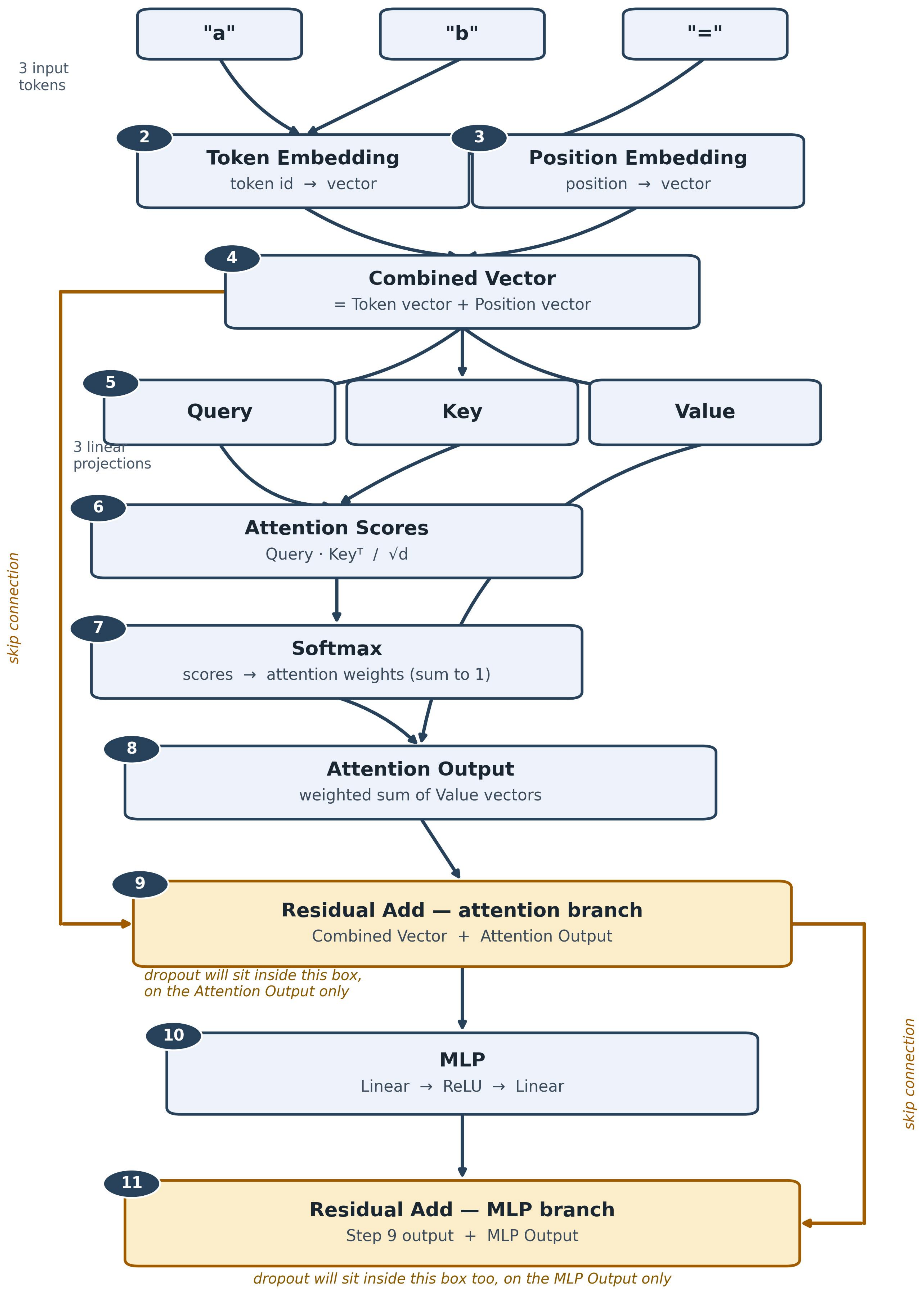


Token 8 (e.g., Target):** Provides Logits (vocab Scores) for each token through the vocabulary. (To get the precise result).



Transformer Forward Pass

Everything that happens BEFORE dropout is applied



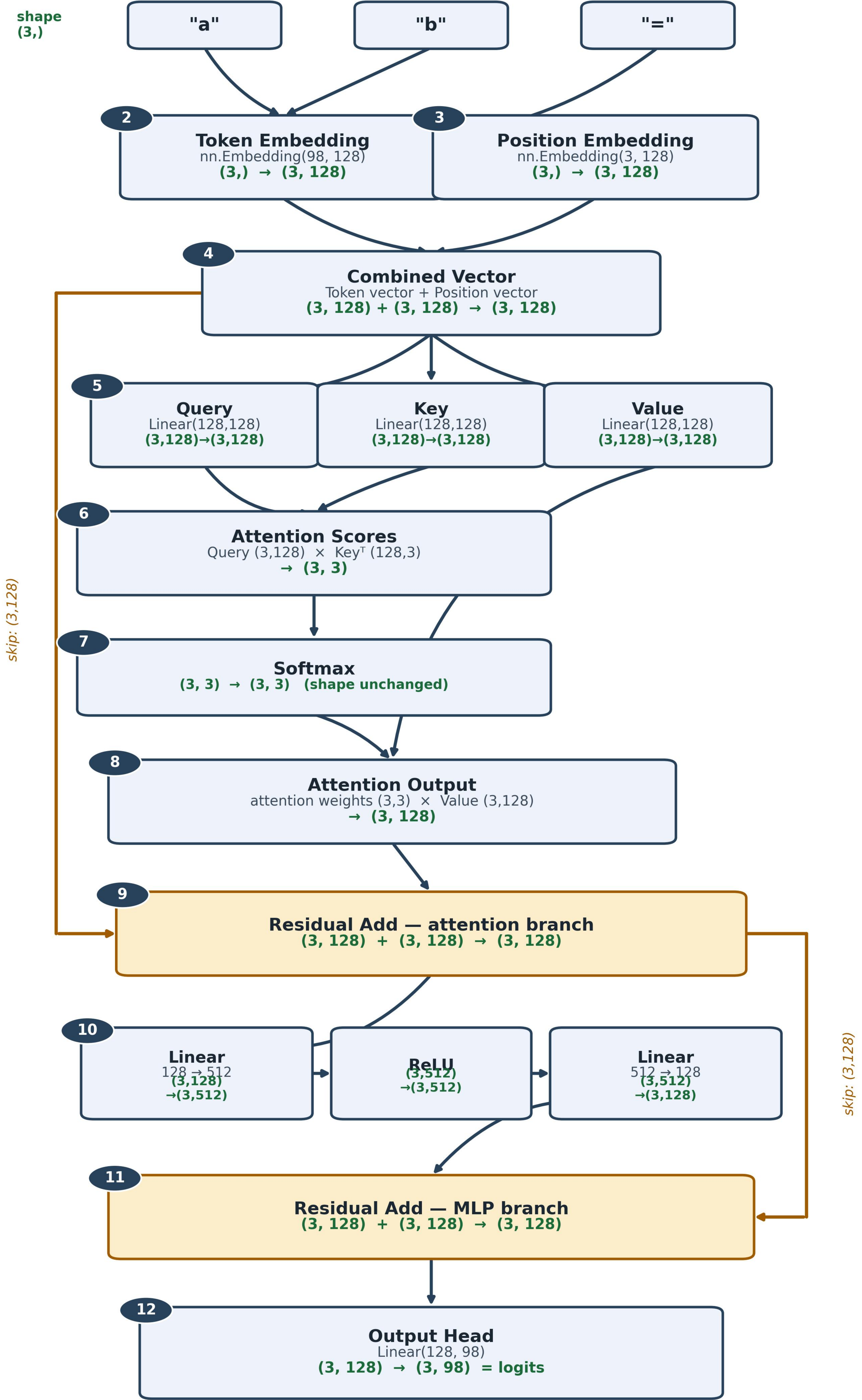
Dropout is not part of this diagram.

It is applied only inside the two orange Residual Add boxes (Step 9 and Step 11) — and only on the newly computed branch, never on the skip connection itself.

Transformer Forward Pass — Shapes

How the matrix shape changes at every layer

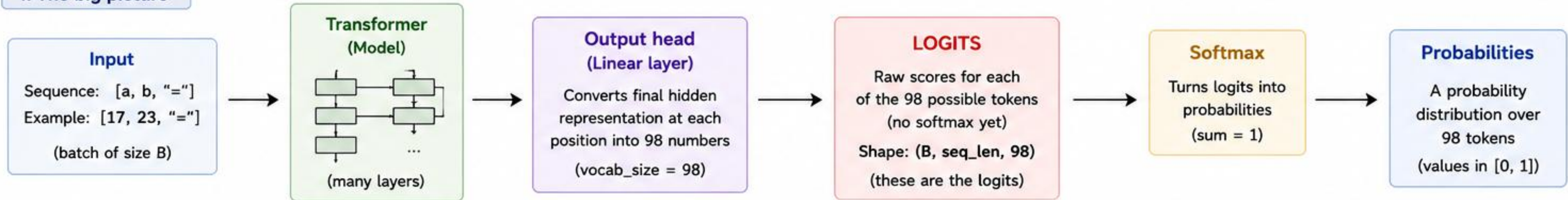
Batch dimension is ignored (batch size = 1). $seq_len = 3$, $d_model = 128$, $vocab_size = 98$



What Exactly Is a Logit?

A logit is the **raw, unnormalized score** the model outputs for each class before turning them into probabilities.

1. The big picture



2. What do logits look like? (example)

Assume batch size $B = 2$ and sequence length = 3
(positions: 0 $\rightarrow$ a, 1 $\rightarrow$ b, 2 $\rightarrow$ "=")

Batch	Pos 0 (a)	Pos 1 (b)	Pos 2 ("=")
Example 1	[17,	23,	"="]
Example 2	[5,	9,	"="]



Logits tensor from model.output_head Shape: (2, 3, 98)			
	Pos 0 (a)	Pos 1 (b)	Pos 2 ("=")
Batch 0 (Example 1)	[-1.2, 0.7, 2.5, ..., -0.3] (98 numbers) 0 1 2 ... 97	[-1.3, 0.2, -0.5, ..., 0.7] (98 numbers) 0 1 2 ... 97	[-1.2, 0.7, 2.5, ..., -0.3] (98 numbers) 0 1 2 ... 97
Batch 1 (Example 2)	[-0.5, 1.1, 0.3, ..., -2.0] (98 numbers) 0 1 2 ... 97	[-1.0, 0.2, -0.1, ..., 1.7] (98 numbers) 0 1 2 ... 97	[0.4, -0.3, 1.9, ..., -1.2] (98 numbers) 0 1 2 ... 97



At position 2 ("=") for Example 1:

Logits (length = 98)

-0.8	1.7	-1.3	0.0	...	-0.6	2.1	0.3
0	1	2	3	...	95	96	97

- Each of the 98 numbers is a **logit** = a raw score.
- Higher logit $\rightarrow$ the model "prefers" that token.
- These scores can be any real number (negative, zero, or positive).

3. Logits vs Probabilities

Logits (raw scores) (can be any real numbers)	Probabilities (after softmax) (sum = 1)
-0.8 1.7 -1.3 0.0 ... -0.6 2.1 0.3 0 1 2 3 ... 95 96 97	0.005 0.079 0.003 0.020 ... 0.006 0.162 0.030 0 1 2 3 ... 95 96 97

Softmax
(exp + normalize)

Not a probability distribution.
Values don't add up to 1 and can be negative.

This is a valid probability distribution.
All values are in $[0, 1]$ and sum to 1.

Important: CrossEntropyLoss expects **RAW LOGITS** as input (not softmax probabilities).
It applies log-softmax internally in a numerically stable way.

4. Which logits are used for loss?

We care only about position 2 ("=") because that's where the model must predict (a + b) % 97.

Example 1: if $(17 + 23) \% 97 = 40$

True target token = 40

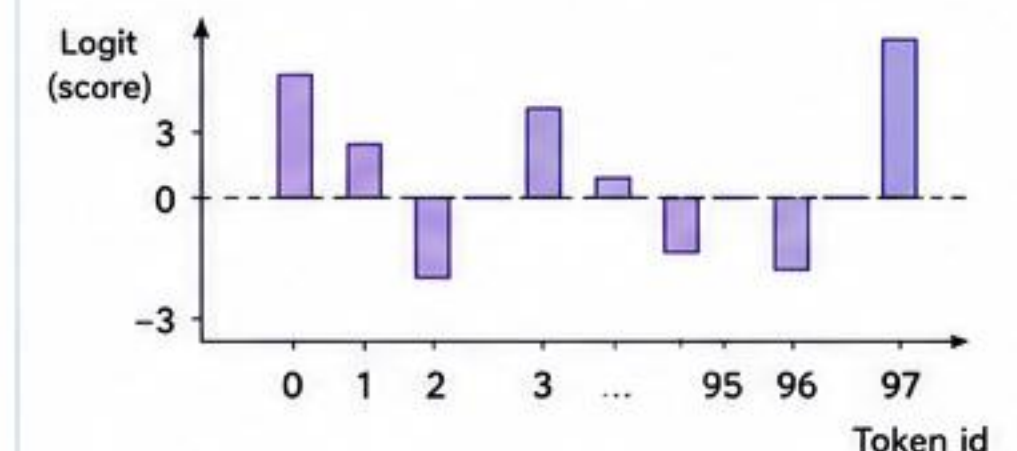
Logits at position 2 for Example 1 (length = 98)

-0.8	1.7	-1.3	...	0.5	...	2.1	0.3
0	1	2		40		96	97

CrossEntropyLoss compares the whole 98-way distribution (after softmax) with the true class (40).

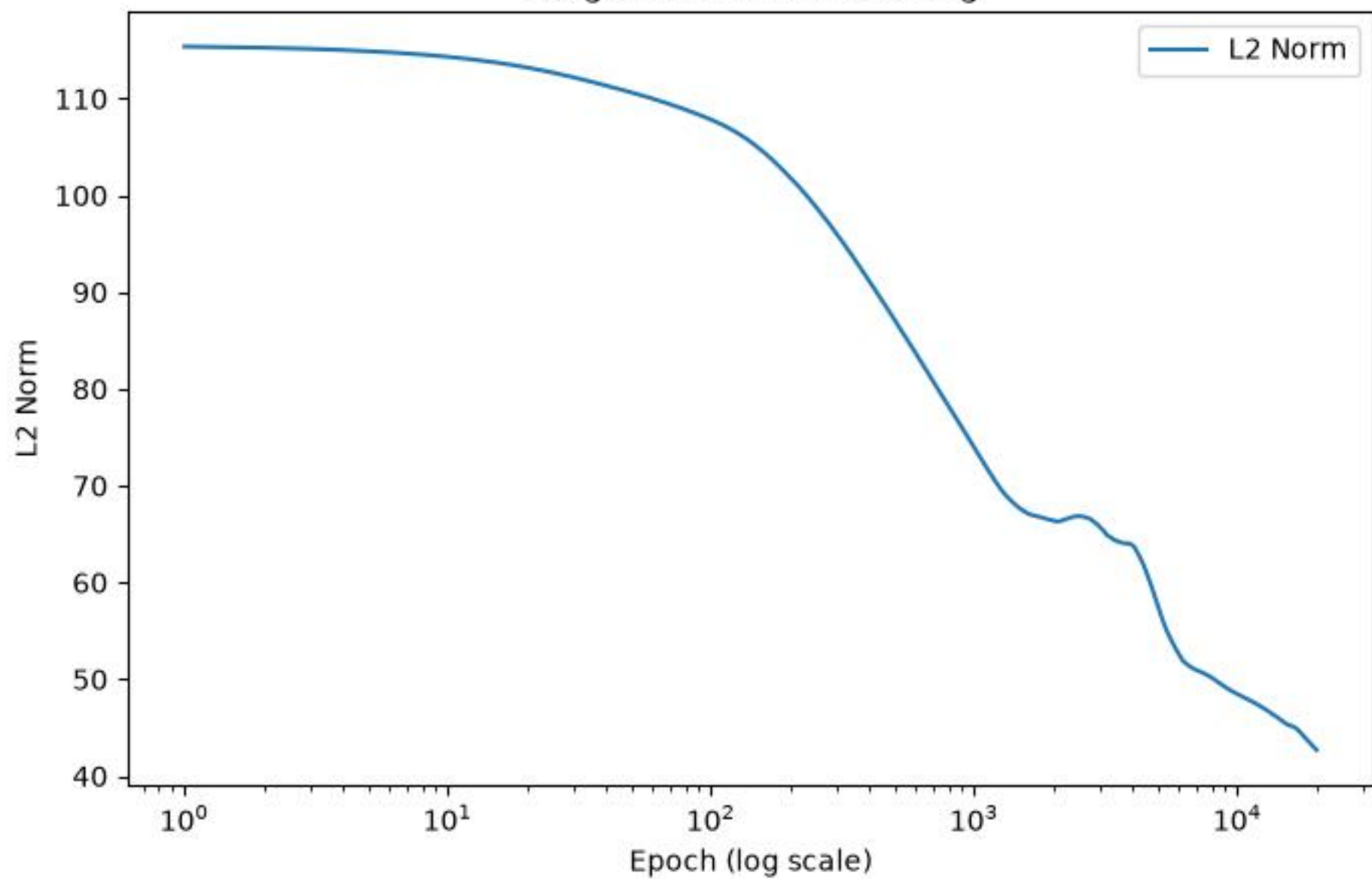
5. Intuition

Think of logits as "confidence scores" before converting to probabilities.

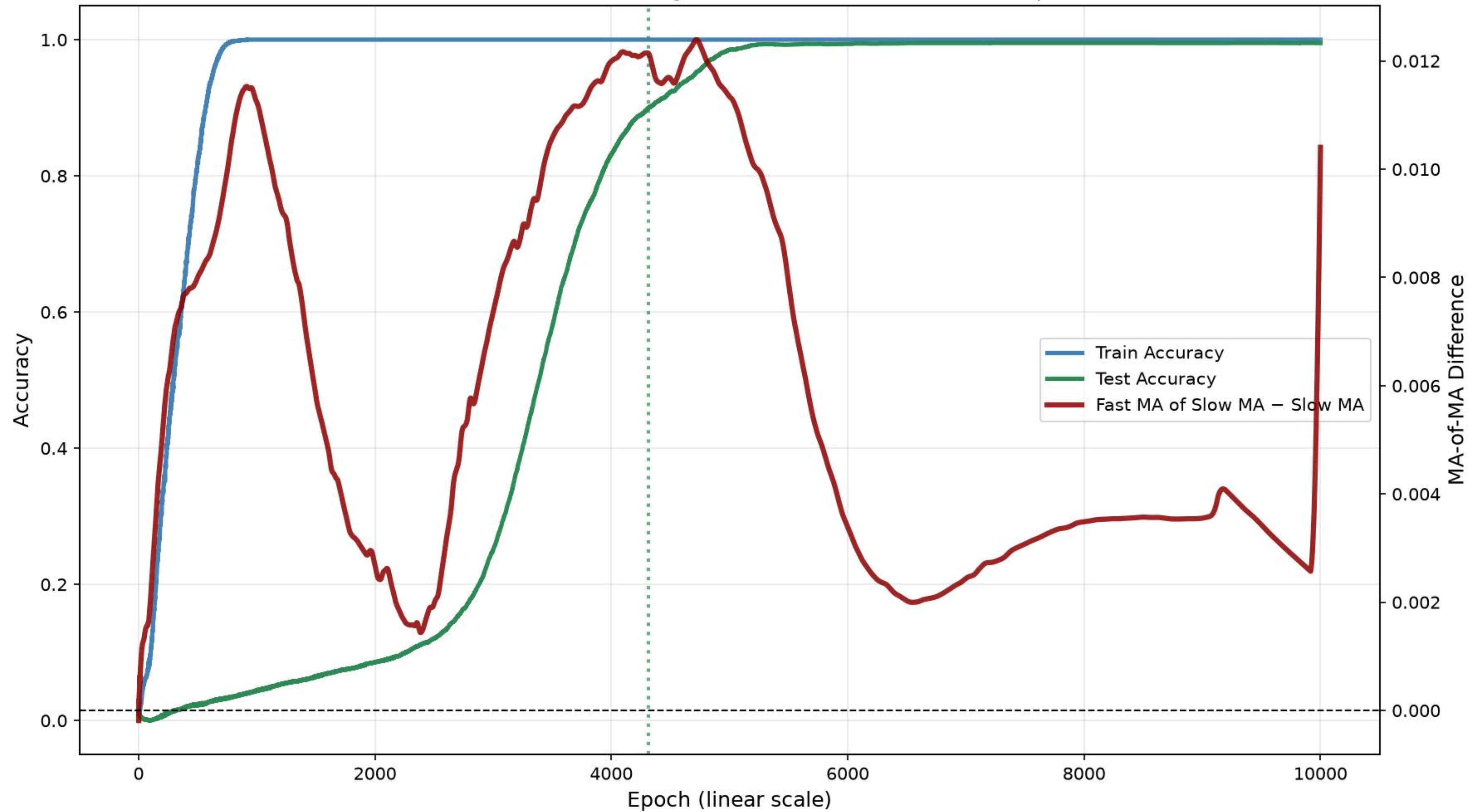


Higher bar = higher logit = more confidence that this token is the correct answer.

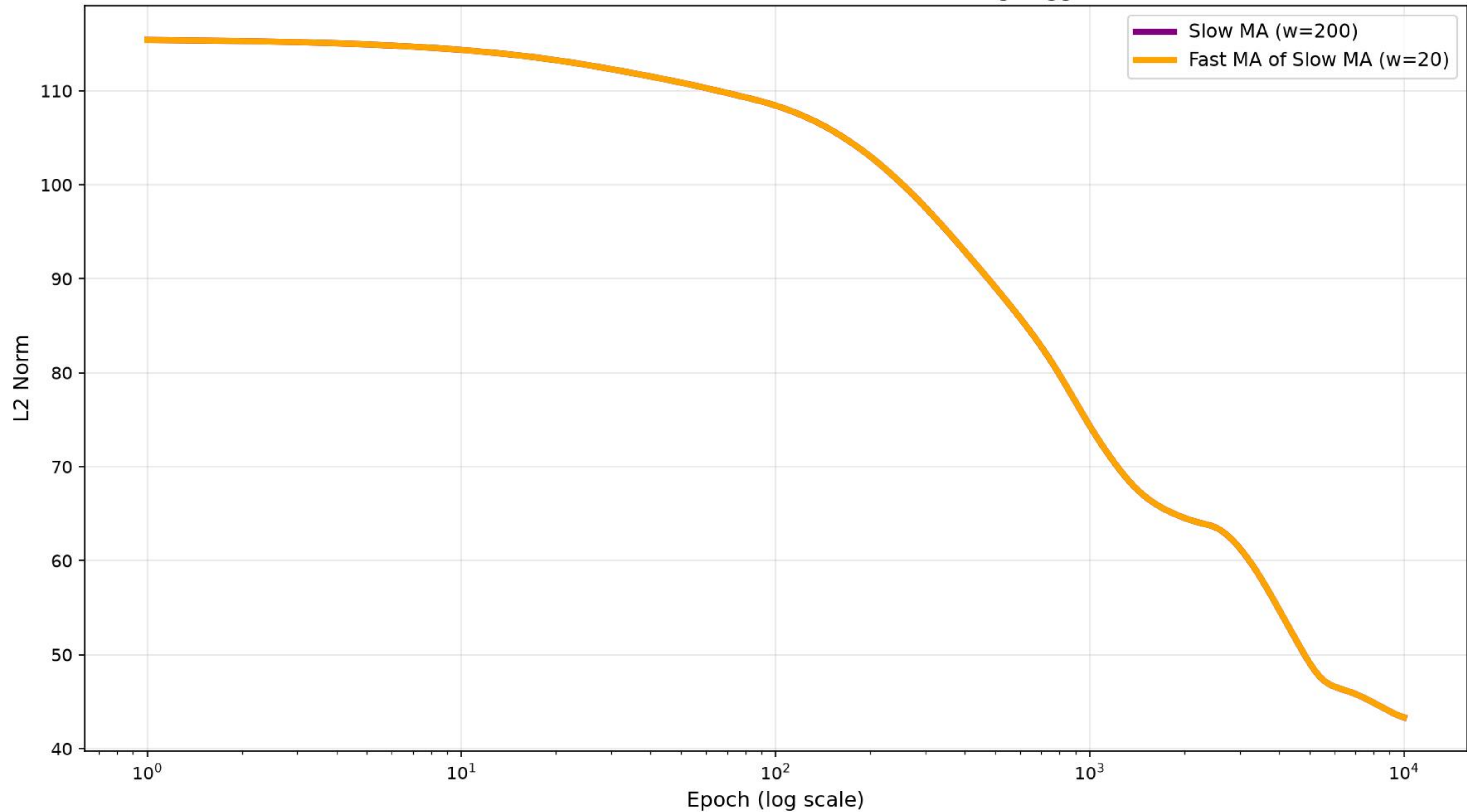
Weight Norm Over Training



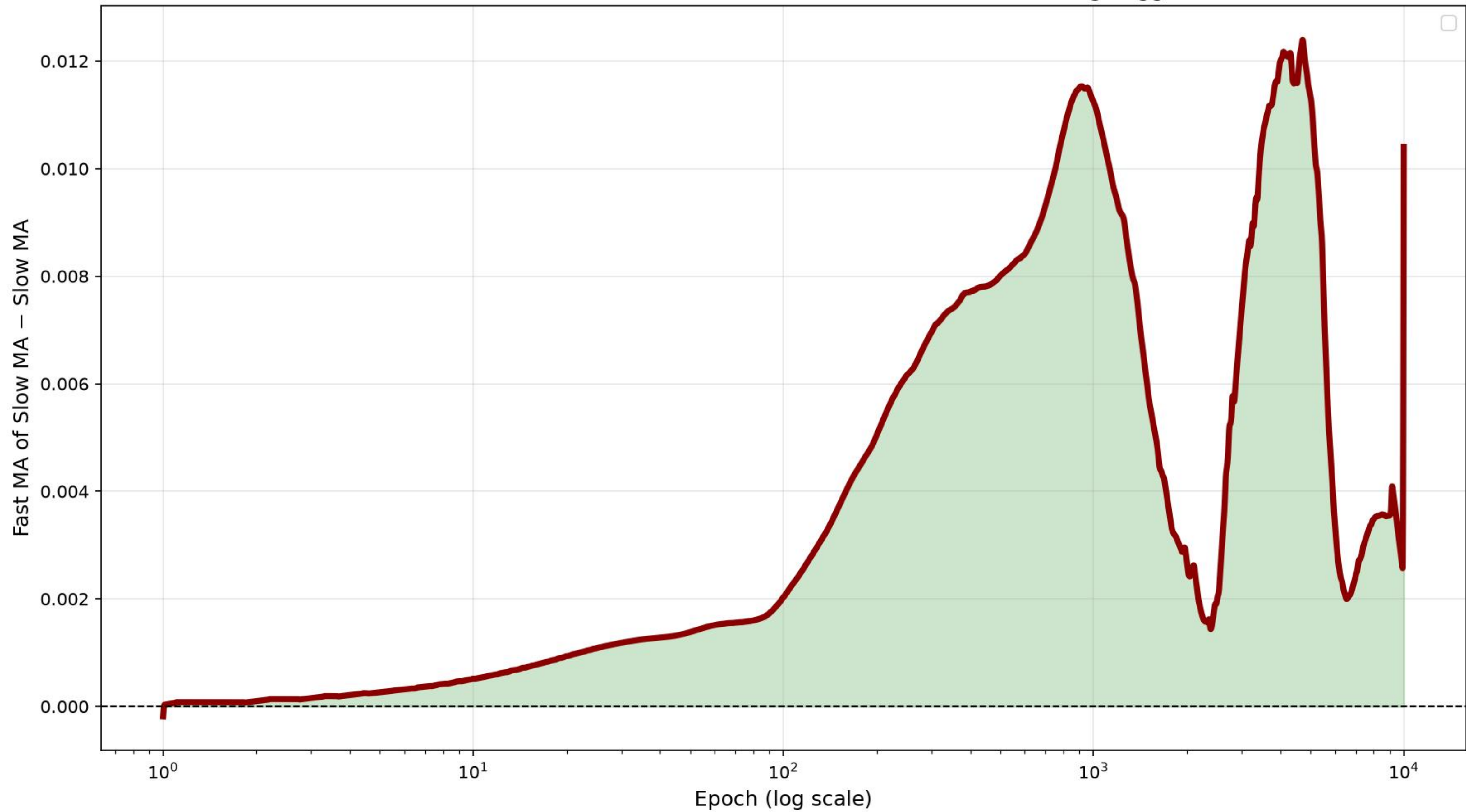
MA-of-MA Difference vs. Grokking Curve (Linear Scale) — Grok epoch 4314



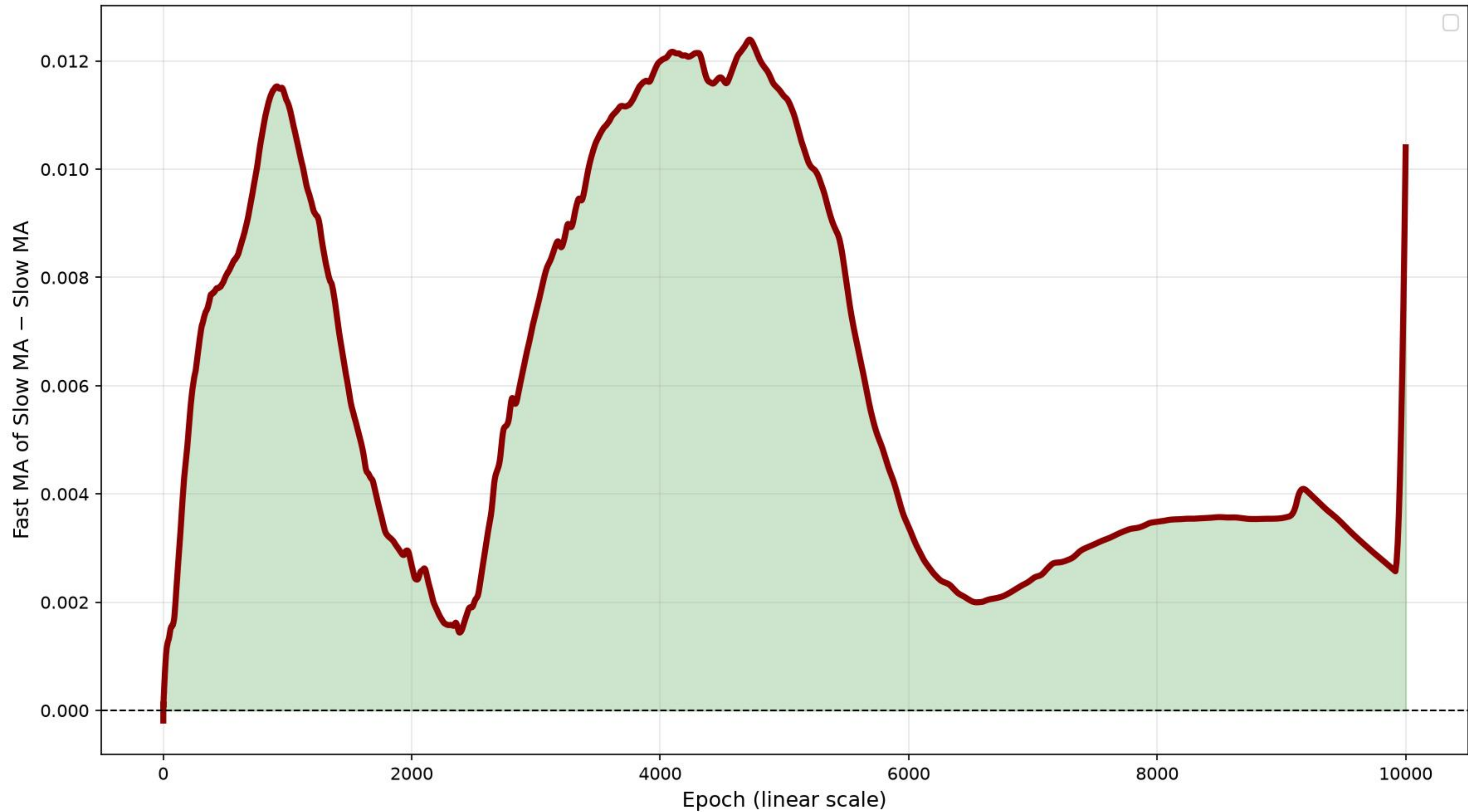
Slow MA vs. Fast MA of Slow MA — Zero-Crossing Trigger



Difference: Fast MA of Slow MA minus Slow MA — Zero-Crossing Trigger



Difference: Fast MA of Slow MA minus Slow MA — Linear Scale



Shadow Model (with LayerNorm) — Grokking Curve vs. Original Plateau

